Large Language Models and the transformer architecture

Basics

Language Models: Definitions

What is a language model?

$P(\text{word}_{n+1} \mid \text{word}_1, \ldots, \text{word}_n)$

- **Definition 1:**

  A statistical model for the probability of a word sequence W

$$P(W) = P(w_1, \ldots, w_n)$$

- **Definition 2:**

  A statistical model for the probability of a word given a context, with context being for example

  - preceding words $P(w_{n+1} \mid w_1, \ldots, w_n)$
  - surrounding words $P(w_i \mid w_1, \ldots w_{i-1}, w_{i+1}, \ldots, w_n)$

Language Models: Definitions

What is a language model?

$P(\text{word}_{n+1} \mid \text{word}_1, \ldots, \text{word}_n)$

- **Aim:**

  Re-create natural language

  (new sequences of language that could pass as natural language)

Language Models: Simple example

A simple language model: Bigram frequency model

- Take a corpus of natural language

```
Alice
was
beginning
to
get
very
tired
of
sitting
by
her
sister
on
the
bank,
and
of
having
nothing
to
do:
```

Language Models: Simple example

A simple langauge model: Bigram frequency model

- Take a corpus of natural language
- Count bigram frequencies

word	word2	Freq
<END>	you	36
she	said	35
with	the	35
there	was	35
the	dormouse	34
on	the	34
that	she	33
was	a	31
i	dont	31
march	hare	31
and	then	31

Language Models: Simple example

A simple langauge model: Bigram frequency model

- Take a corpus of natural language
- Count bigram frequencies
- The probability a word

  given the preceding word in a sequence is

$$P(w_i|w_{i-1}) = \frac{\text{freq}(w_{i-1}w_i)}{\text{freq}(w_{i-1})}$$

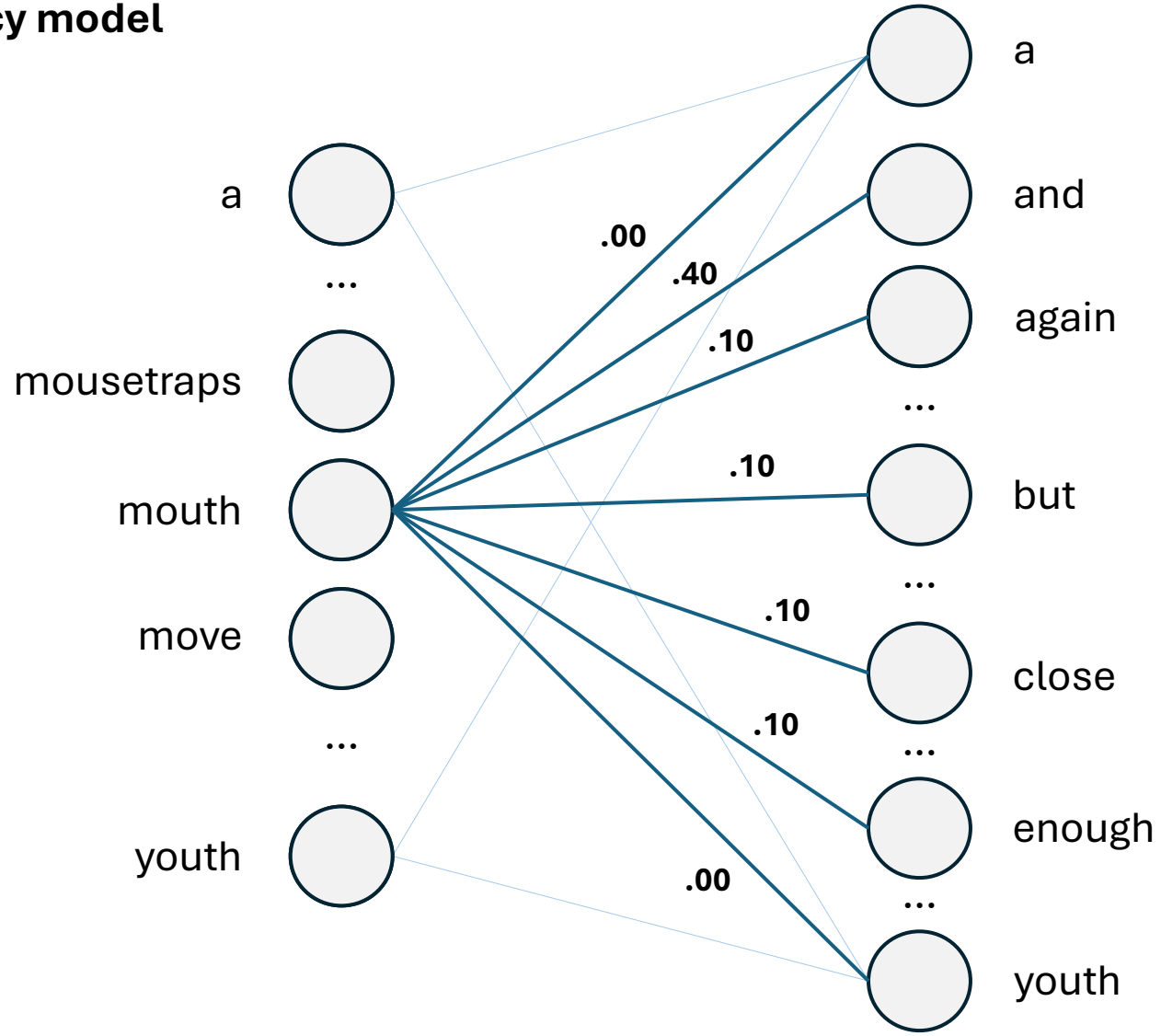
word	word2	Freq	prob
<END>	you	36	0.02214022
she	said	35	0.06517691
with	the	35	0.19444444
there	was	35	0.46666667
the	dormouse	34	0.02103960
on	the	34	0.17616580
that	she	33	0.11956522
was	a	31	0.08732394
i	dont	31	0.07769424
march	hare	31	0.93939394
and	then	31	0.03703704

Language Models: Simple example

A simple langauge model: Bigram frequency model

- We can plot this

word	word2	Freq	prob
mouseto	a	1	1.00000000
mousetraps	and	1	1.00000000
mouth	and	4	0.40000000
mouth	again	1	0.10000000
mouth	but	1	0.10000000
mouth	close	1	0.10000000
mouth	enough	1	0.10000000
mouth	open	1	0.10000000
mouth	with	1	0.10000000

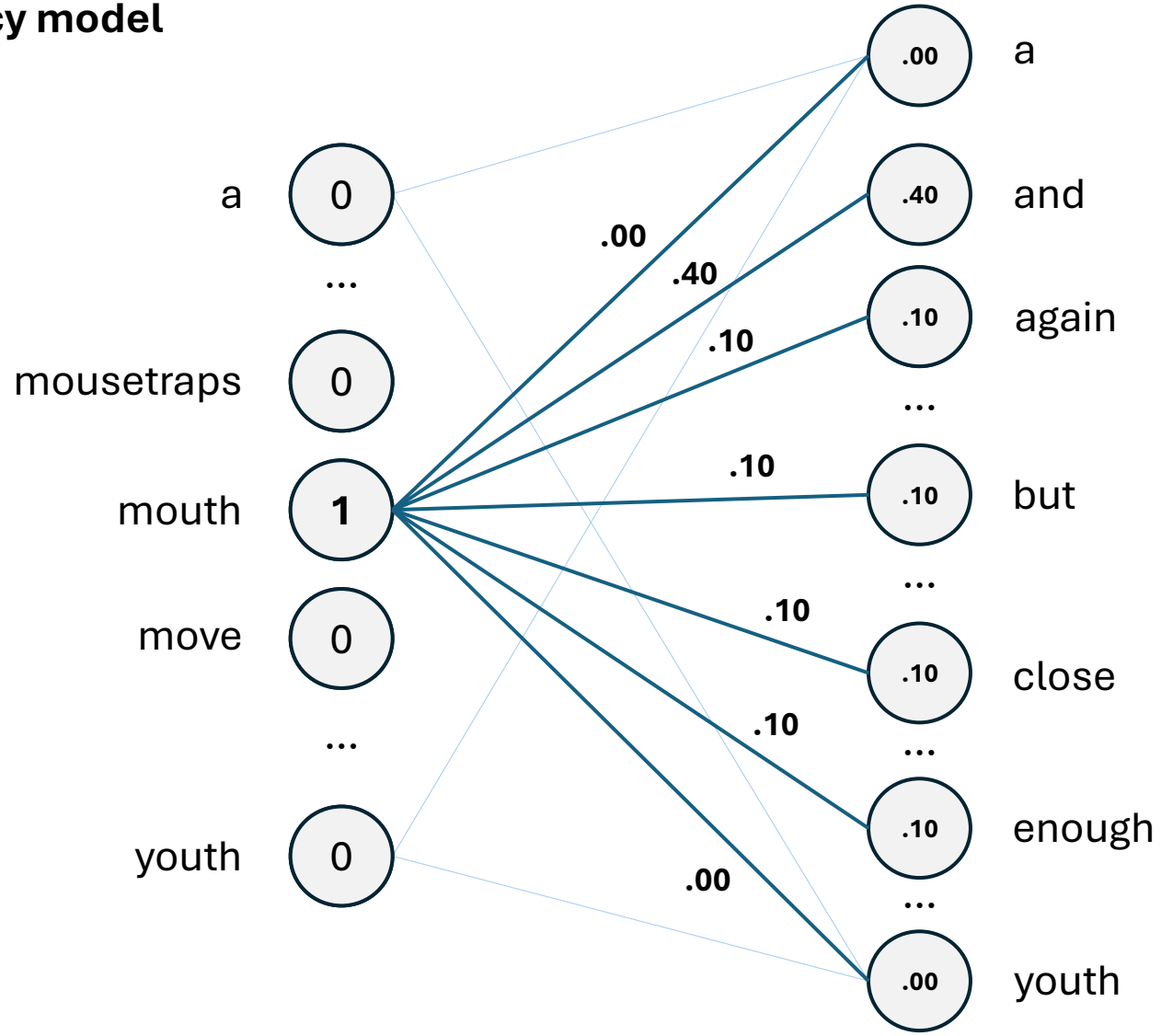


Language Models: Simple example

A simple langauge model: Bigram frequency model

- We can plot this

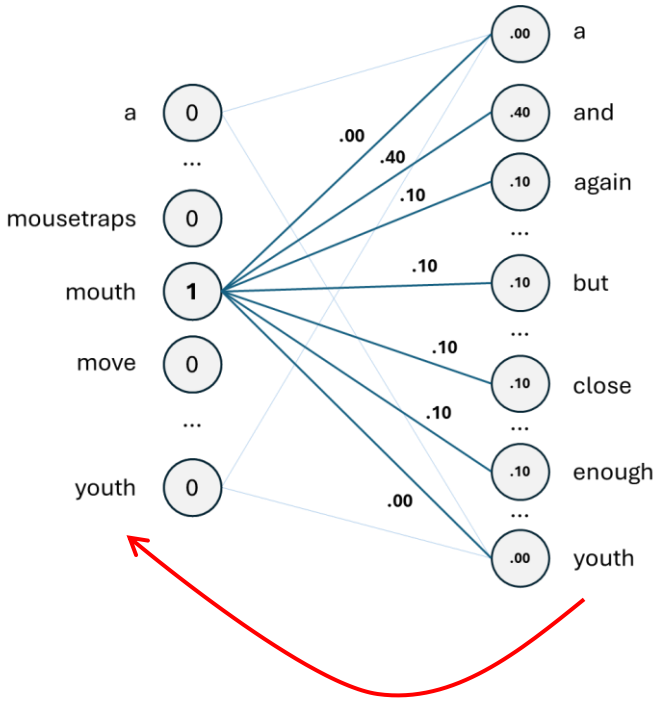
word	word2	Freq	prob
mouseto	a	1	1.00000000
mousetraps	and	1	1.00000000
mouth	and	4	0.40000000
mouth	again	1	0.10000000
mouth	but	1	0.10000000
mouth	close	1	0.10000000
mouth	enough	1	0.10000000
mouth	open	1	0.10000000
mouth	with	1	0.10000000



Language Models: Simple example

A simple langauge model: Bigram frequency model

- We can use this to generate language
  - Start: any word, like **the**
  - Sample words according to their probability
  - Loop until **<END>** is produced



the pleasure in the games going up she knew time ! <END>

the other birds complained that ever so suddenly spread out of sight but why ? <END>

Language Models: Probabilities and output

At this point, we have an understanding of two central components of LMs

- **Probabilities**
  - Of a word
  - Of a word given an input word

- **Output**

Language Models: Probabilities and output

At this point, we have an understanding of two central components of LMs

- **Probabilities**
  - Of a word
  - Of a word given an input word

  - Using basic probability theory, we can use this to calculate probabilities of word sequences [given an input sequence]
  - Probabilities are often converted into **surprisal**

    (higher surprisal: lower probability)

$$\text{surprisal} = -\log(p)$$

Language Models

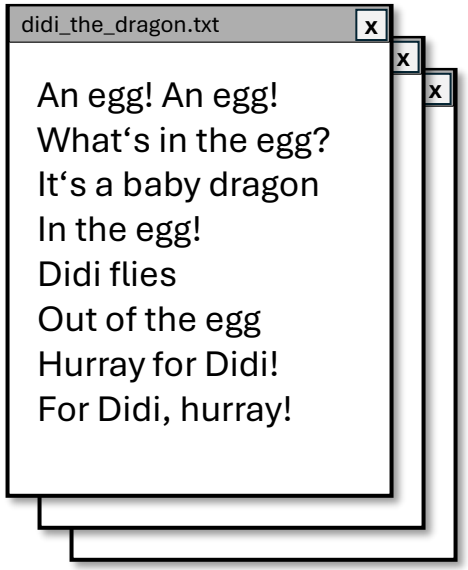
Neural networks and word embeddings

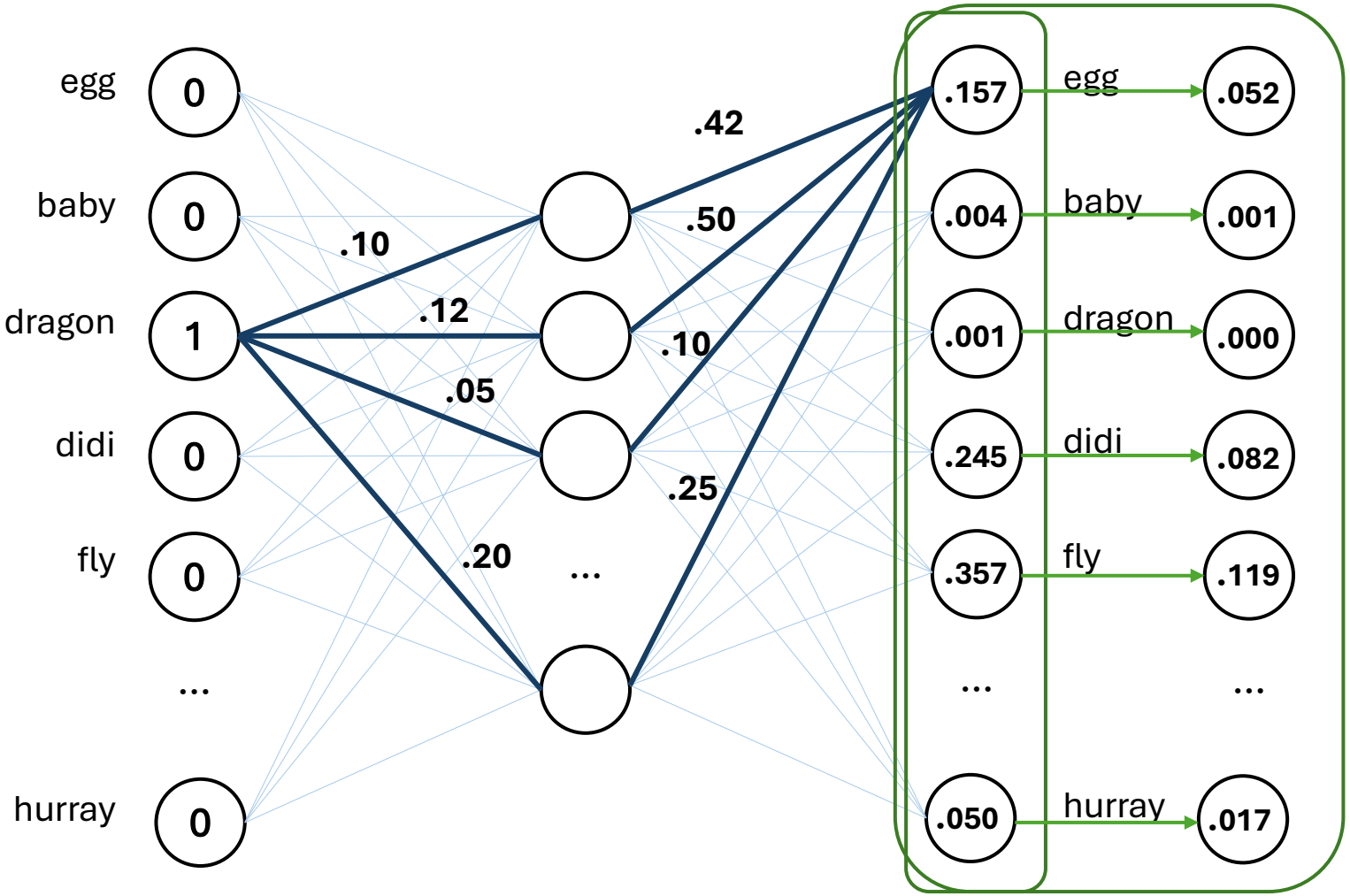
Language Models: Neural networks

Categorical outcomes: Softmax

softmax:

$$p = \frac{e^{\text{activation}}}{\sum e^{\text{activation}}}$$

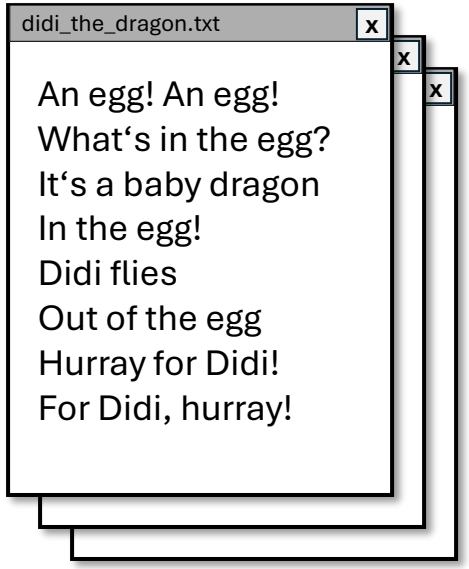


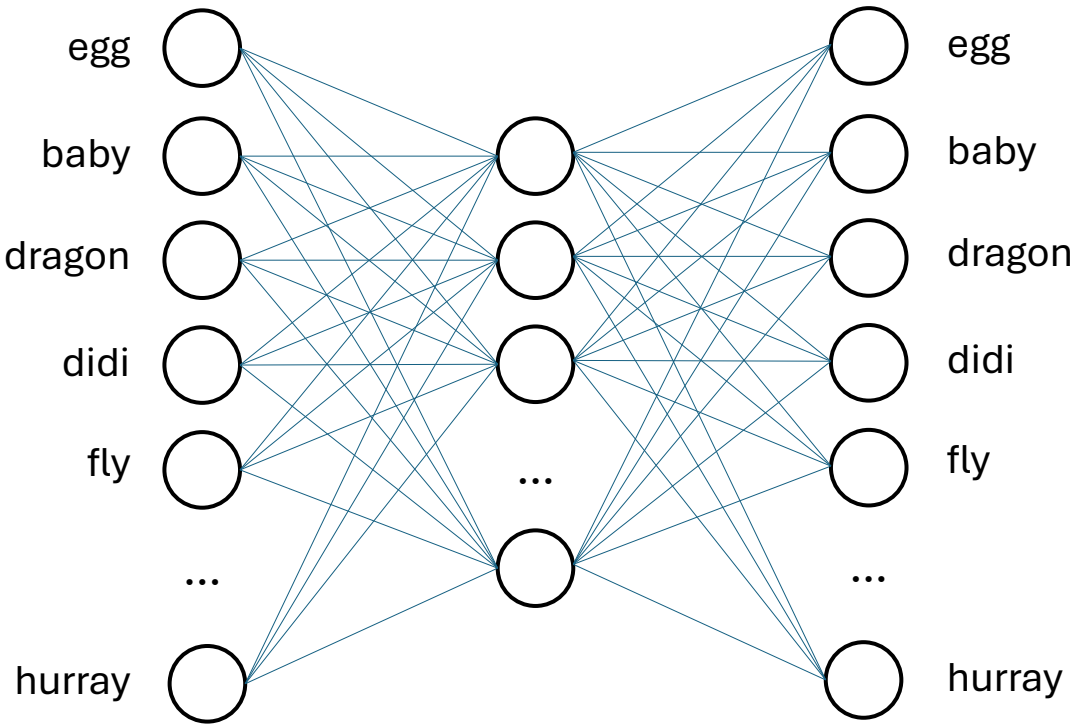


Training the neural network

Learning the weights

Start with random weights





Language Models: Neural networks

Training the neural network

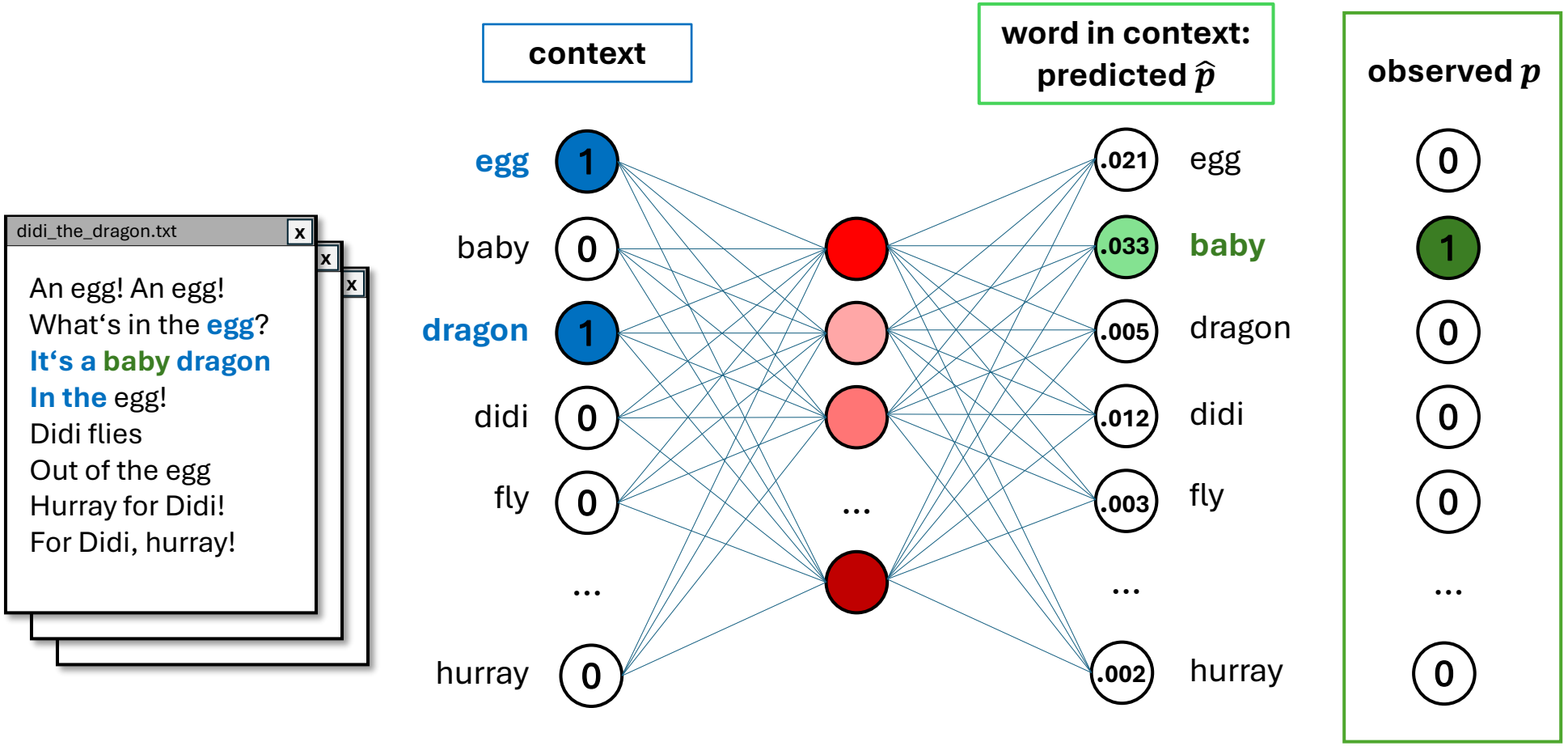
Learning the weights

Update the weights based on loss

Larger loss → larger changes

Calculate loss = prediction error

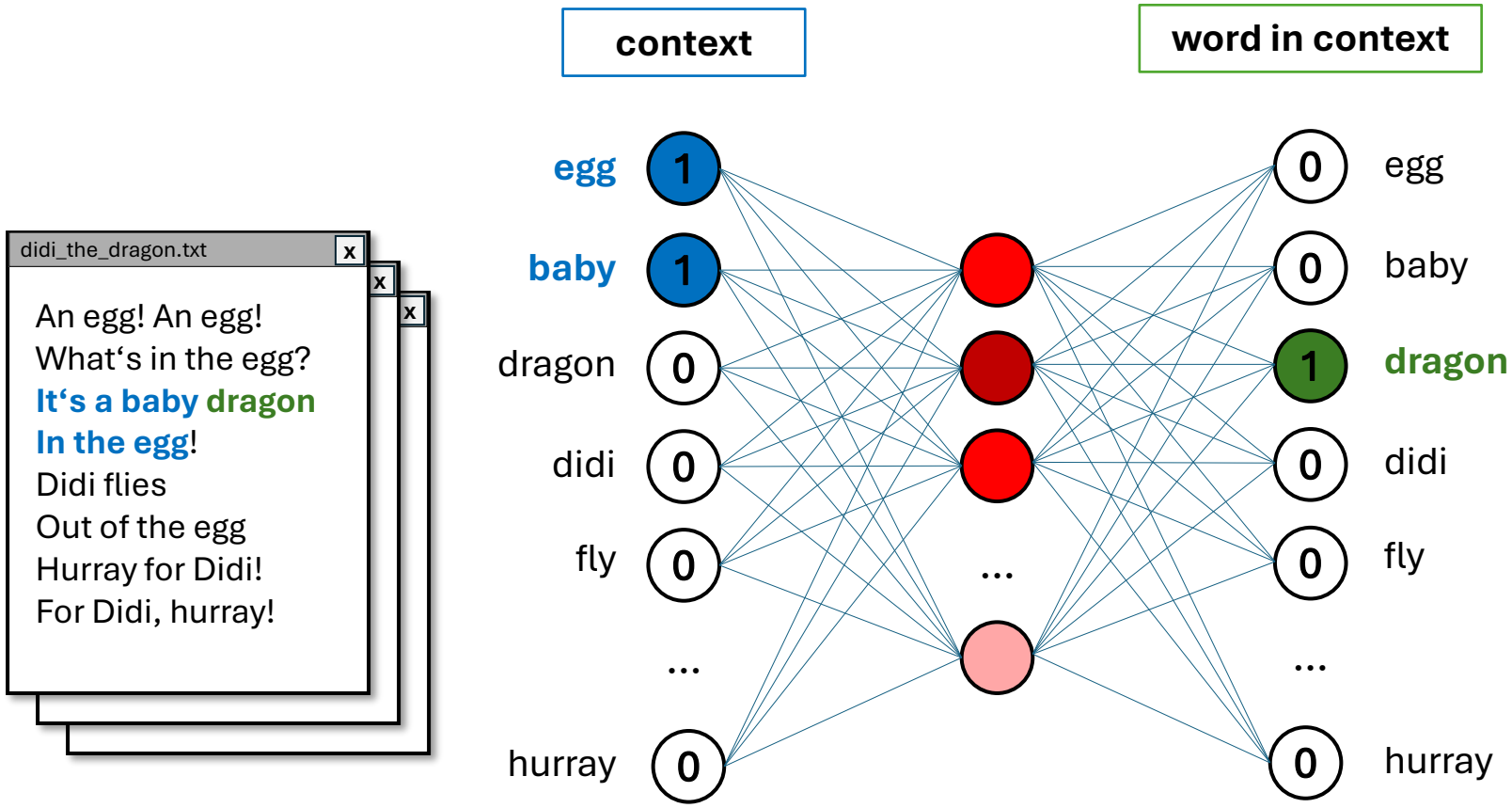
$$\text{loss} = \Delta(\hat{p}, p)$$



Training the neural network

Learning the weights

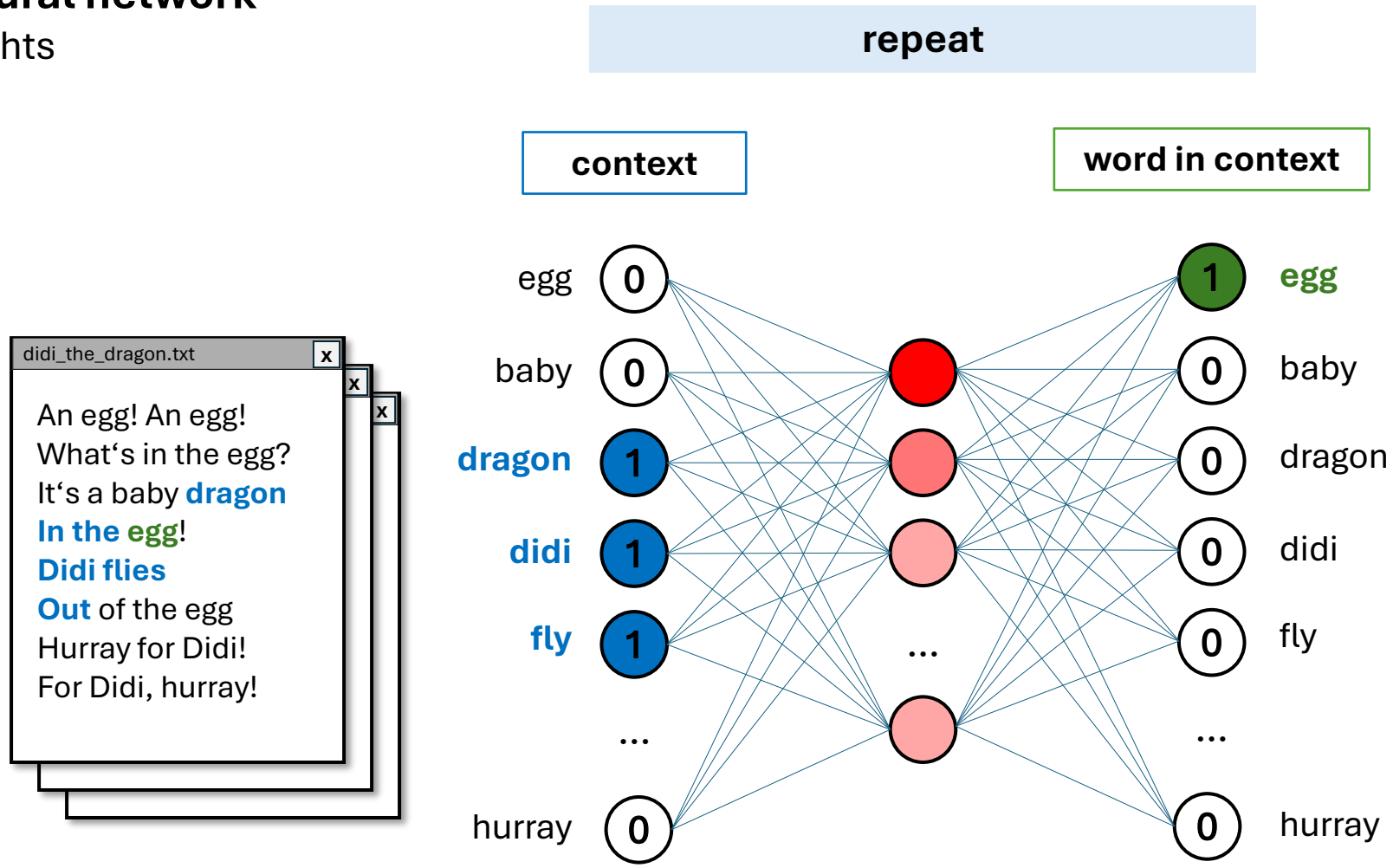
Go through the corpus



Language Models: Neural networks

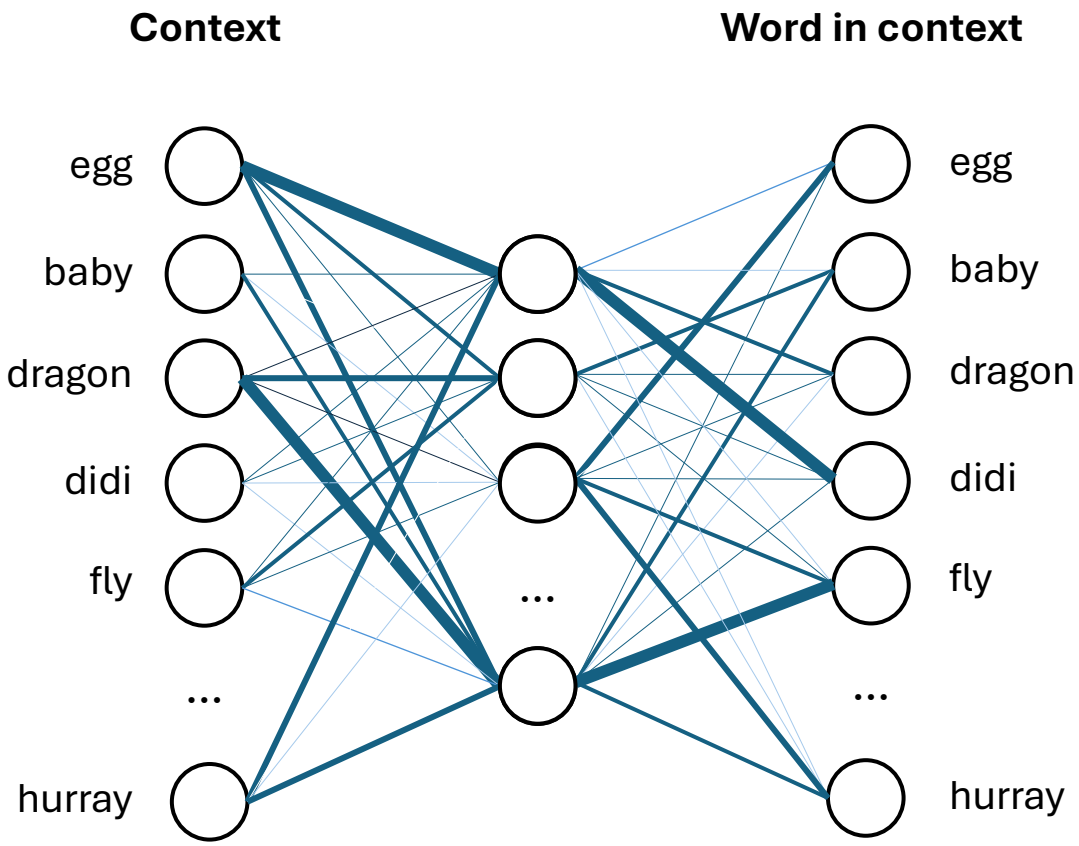
Training the neural network

Learning the weights

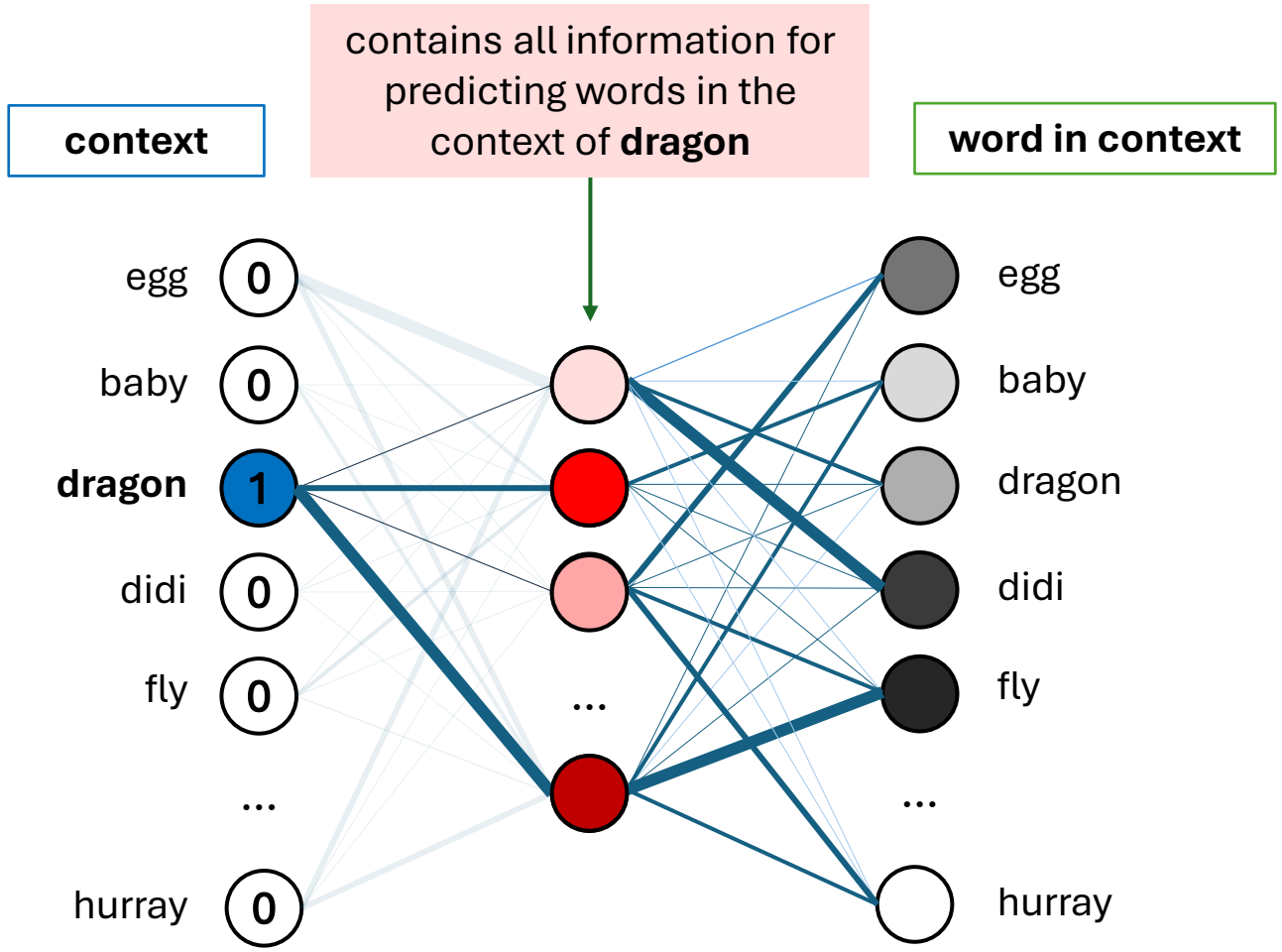


Language Models: Neural networks

After training:
Weights are fixed



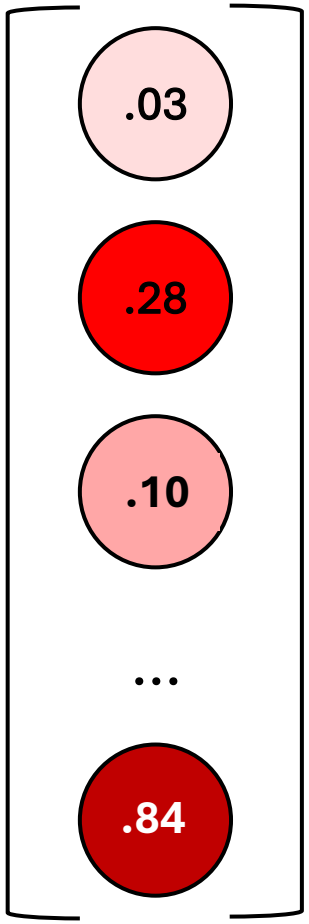
Language Models: Word embeddings



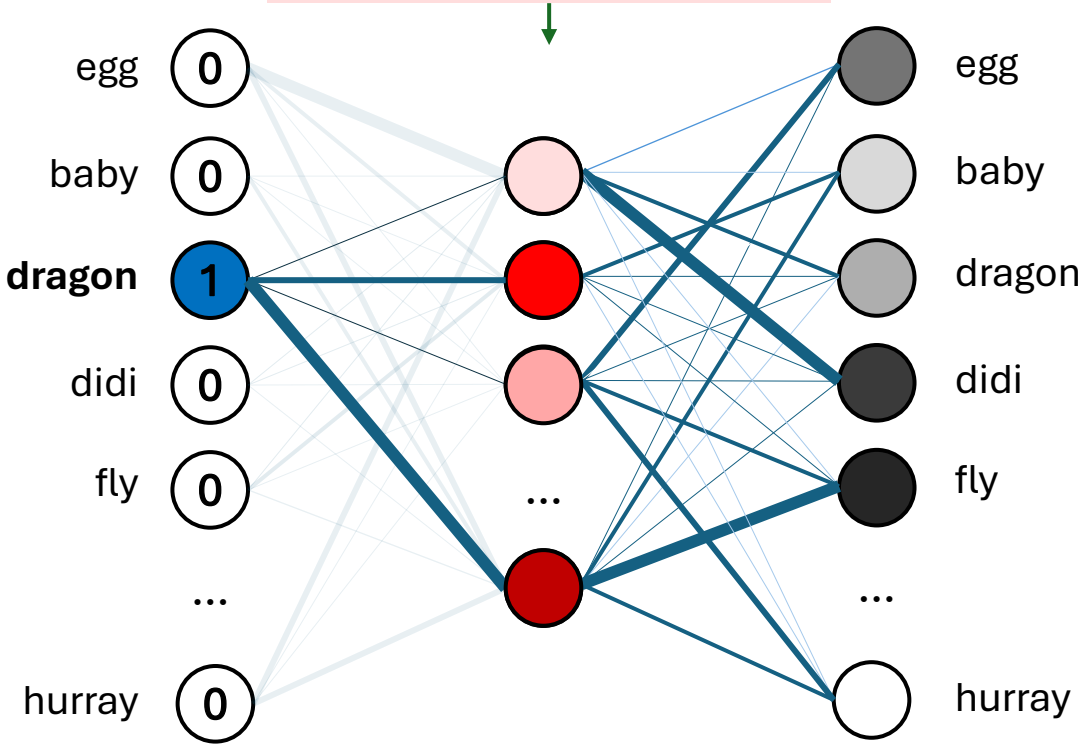
Language Models: Word embeddings

word embedding as a semantic vector representation

$\overrightarrow{\text{dragon}} =$



Word embedding: A **vector** containing all information for predicting words in the context of the target word (here: **dragon**)



$P(word \mid \mathbf{dragon})$

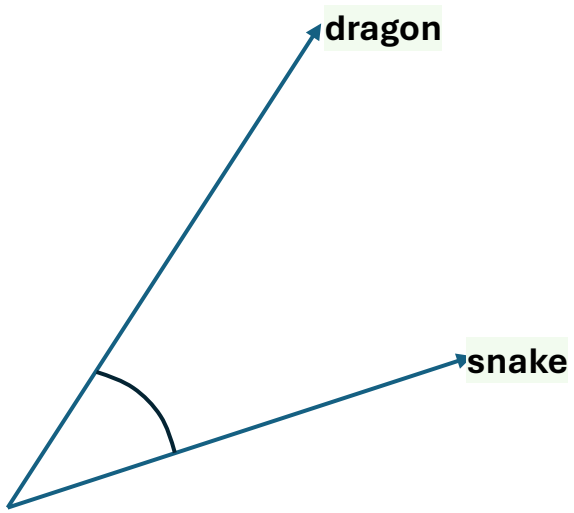
Language Models: Word embeddings

Similarity metrics for word embeddings:

Cosine similarity

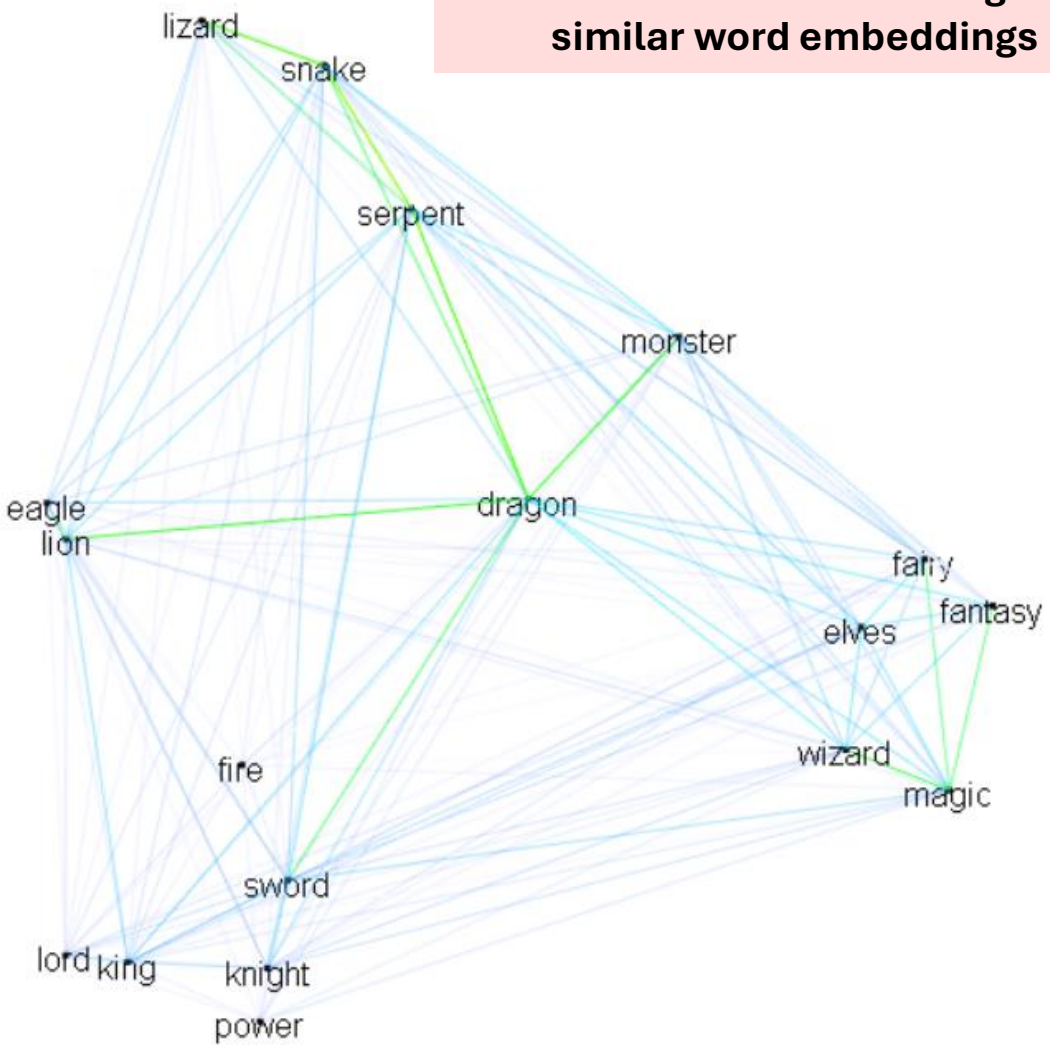
$$\mathrm{sim}(\mathrm{a},\mathrm{b}) = \cos(\mathrm{a},\mathrm{b}) = \frac{\sum_{i=1}^{n} a_i b_i}{\sum_{i=1}^{n} a_i^2 \cdot \sum_{i=1}^{n} b_i^2}$$

$$\overrightarrow{\mathrm{dragon}} = \begin{bmatrix} .03 \\ .28 \\ .10 \\ \dots \\ .84 \end{bmatrix}$$

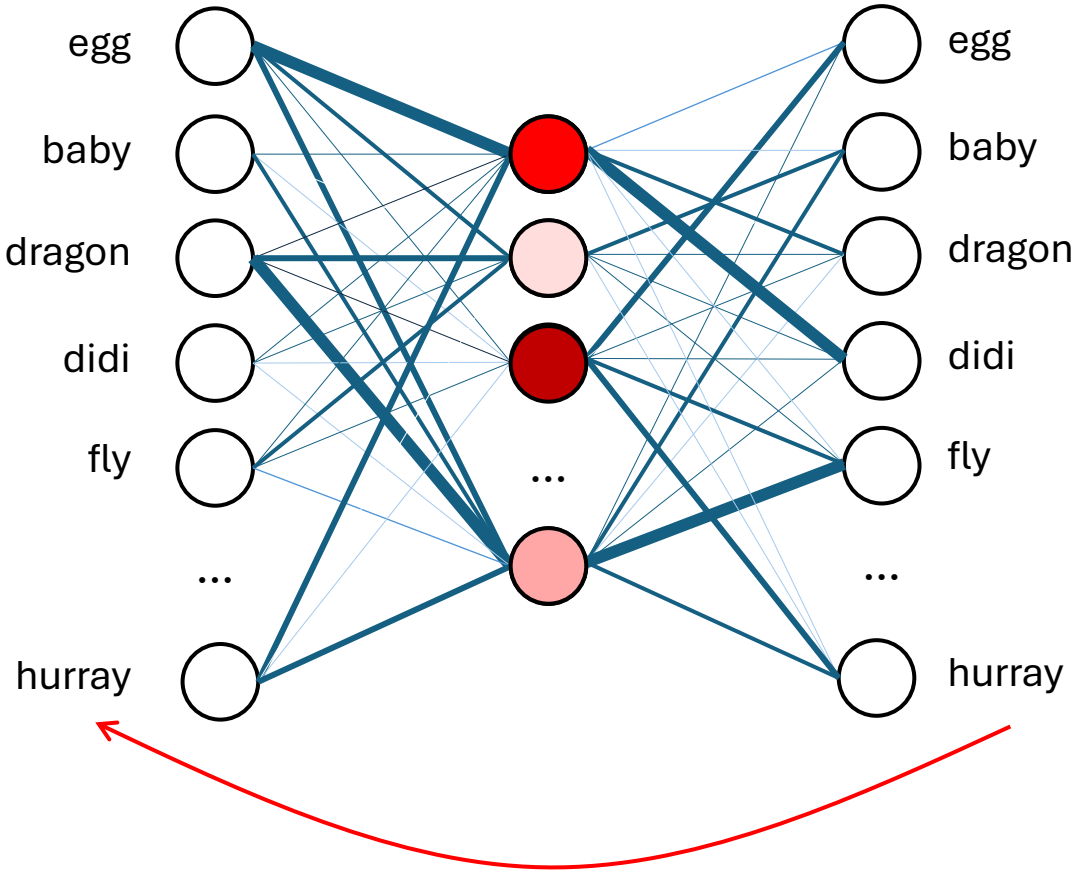


Language Models: Word embeddings

words with **similar meaning** have **similar word embeddings**



Language Models: Word embeddings



We can again use this to generate language

It has the three core components of LLMs:

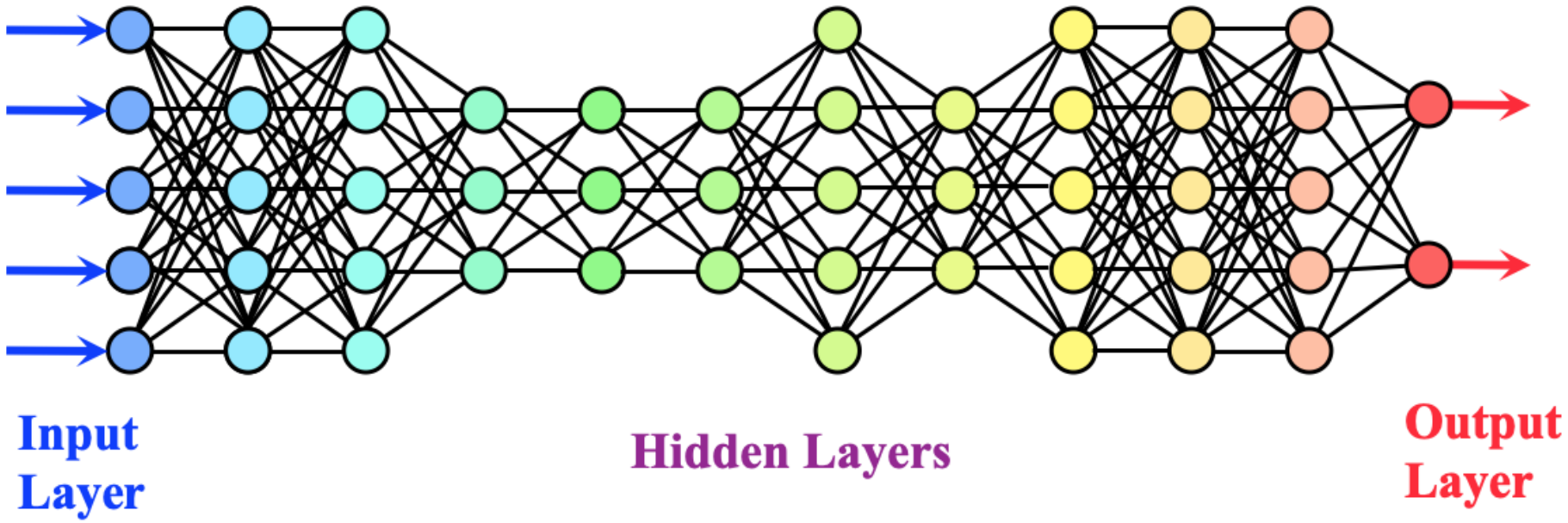
- **output**
- **probabilities**
- **embeddings**

General assessment of these **fixed-window neural language models**:

- useful embeddings for lexical semantics
- probabilities and output still not great

Transition: Deep neural networks

We can have more than one hidden layer



... and add all kinds of extra components

Large Language Models

The transformer architecture

Large Language Models: Transformer overview

fixed-window neural language models

output distribution

$$\hat{y} = \text{softmax}(W_2 h)$$

hidden layer

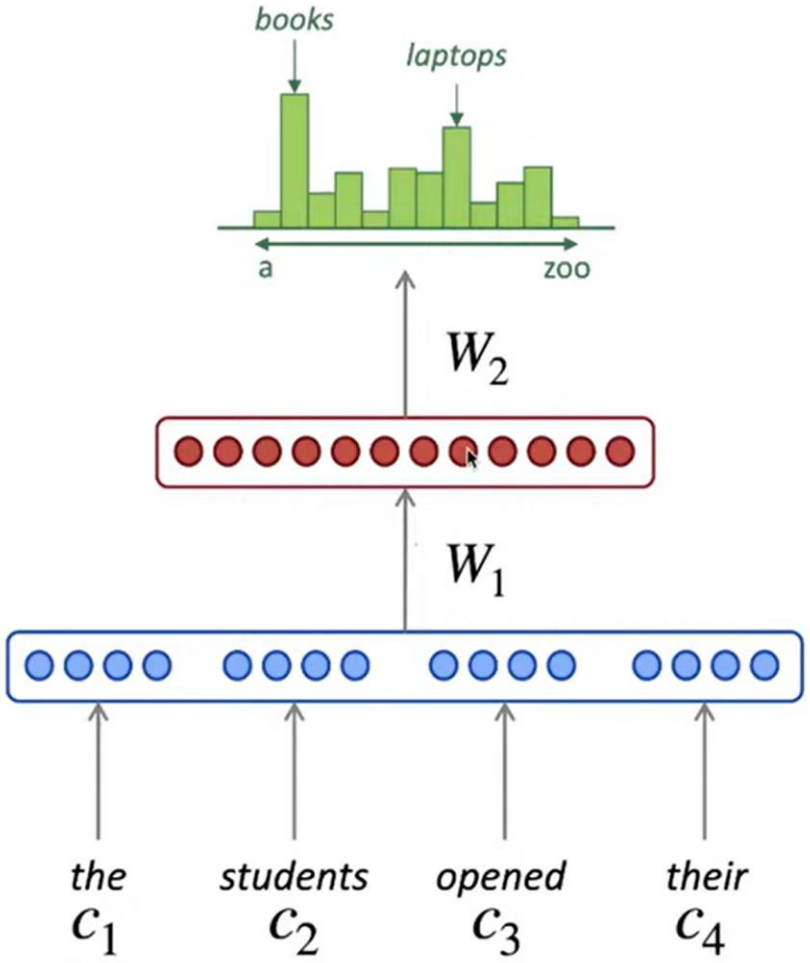
$$h = f(W_1 x)$$

concatenated word embeddings

$$x = [c_1; c_2; c_3; c_4]$$

words / one-hot vectors

$$c_1, c_2, c_3, c_4$$



UMass CS685 S24 (Advanced NLP) 3: https://www.youtube.com/watch?v=qESAY4AE2IQ

Large Language Models: Transformer overview

2017

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

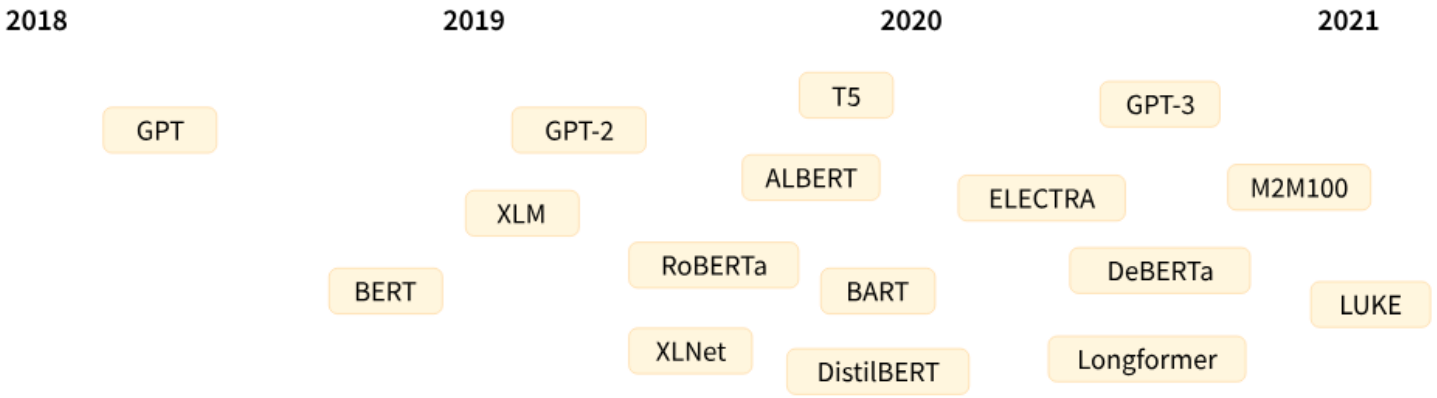
Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

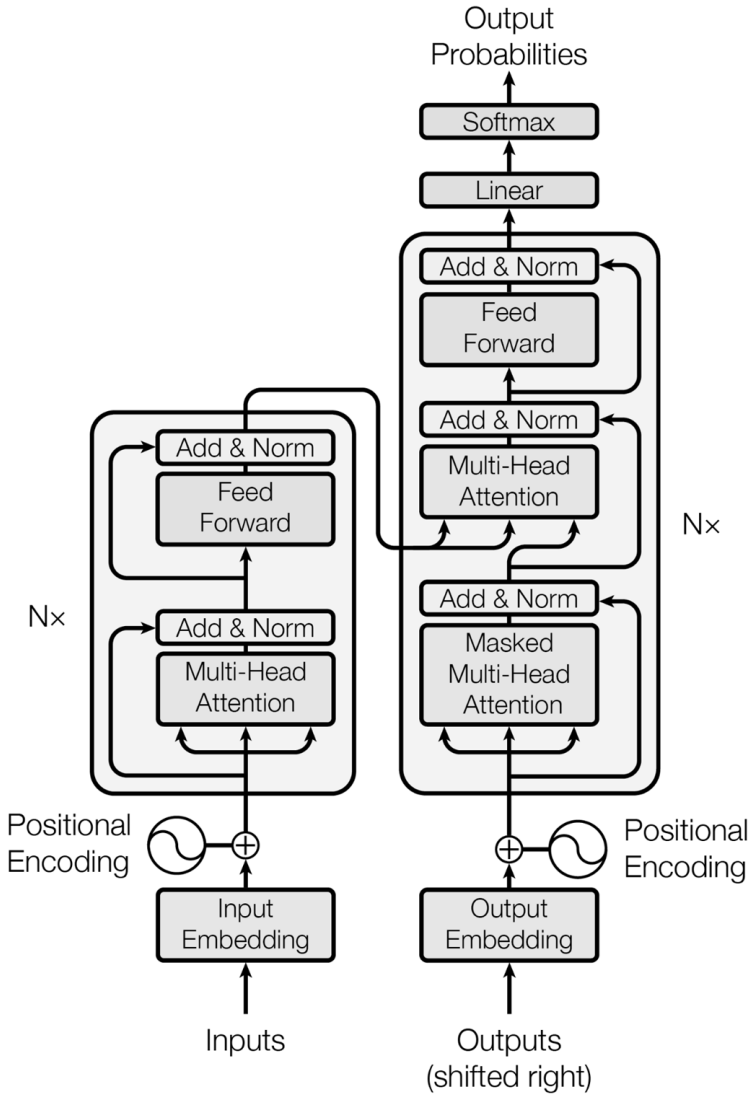
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com



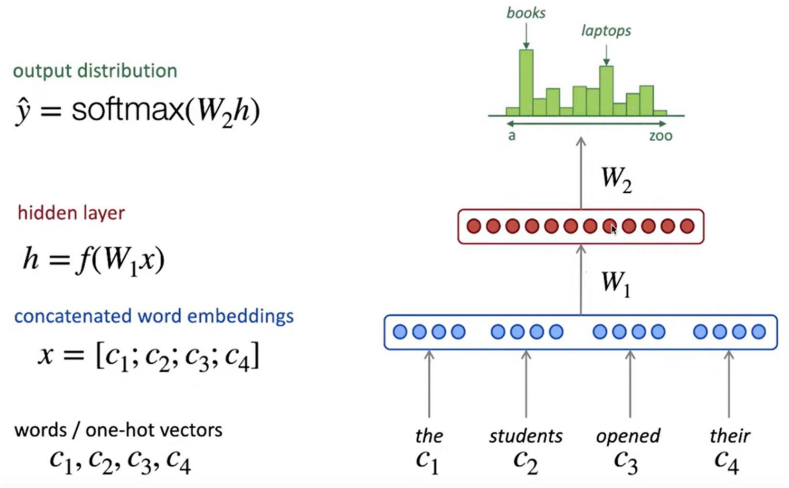
transformer model



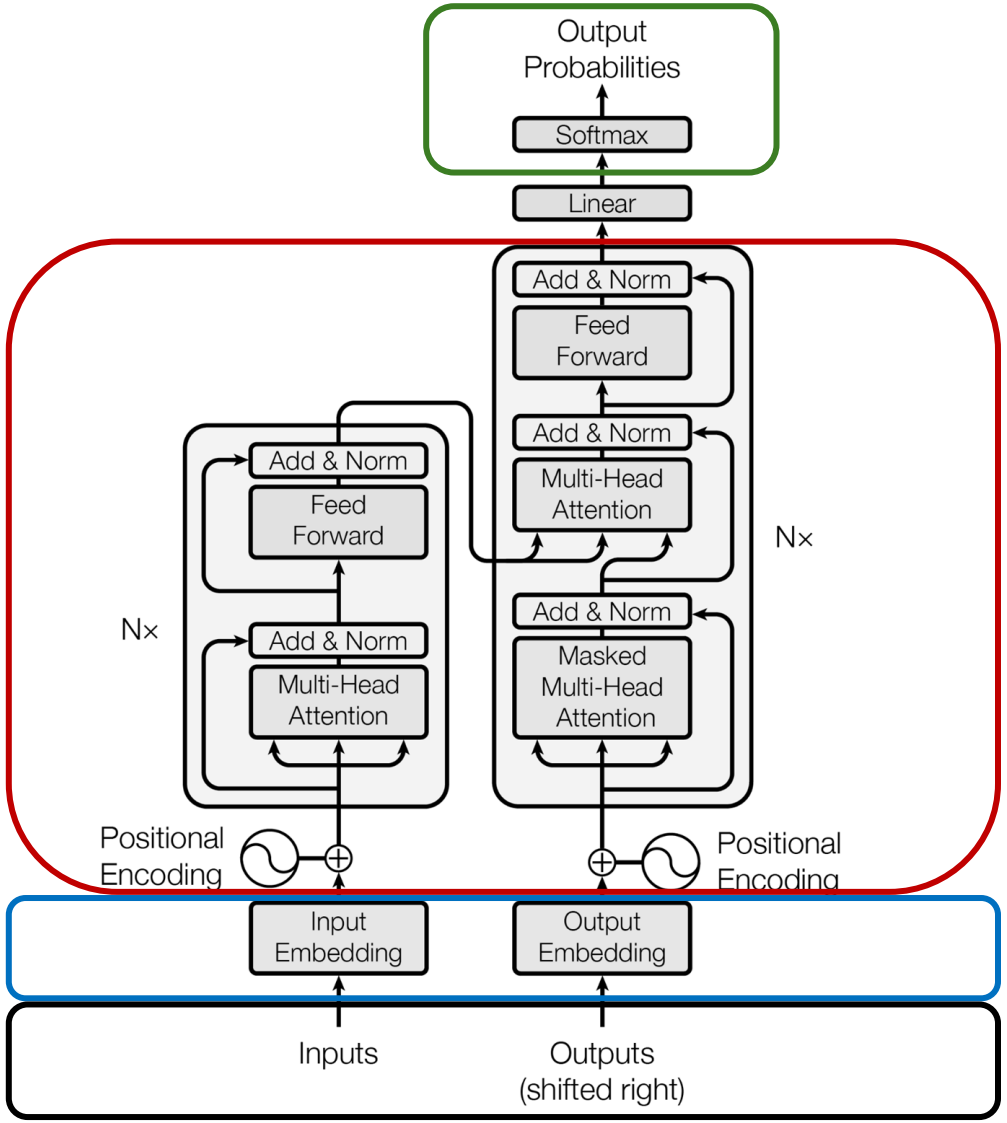
https://huggingface.co/learn/llm-course/en/chapter1/4

Large Language Models: Transformer overview

fixed-window neural language models



transformer model

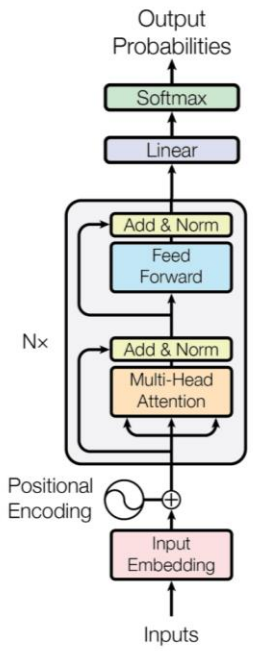


Let‘s have a look at the additional components

UMass CS685 S24 (Advanced NLP) 3: https://www.youtube.com/watch?v=qESAY4AE2IQ

Large Language Models: Transformer overview

BERT*



Encoder-only

Get accurate representation of input

Transformer

Encoder **Decoder**

GPT*

Decoder-only

Generate text

*actual model architectures differ in details

Encoder: Training objective

Training objective

Output:
Masked token

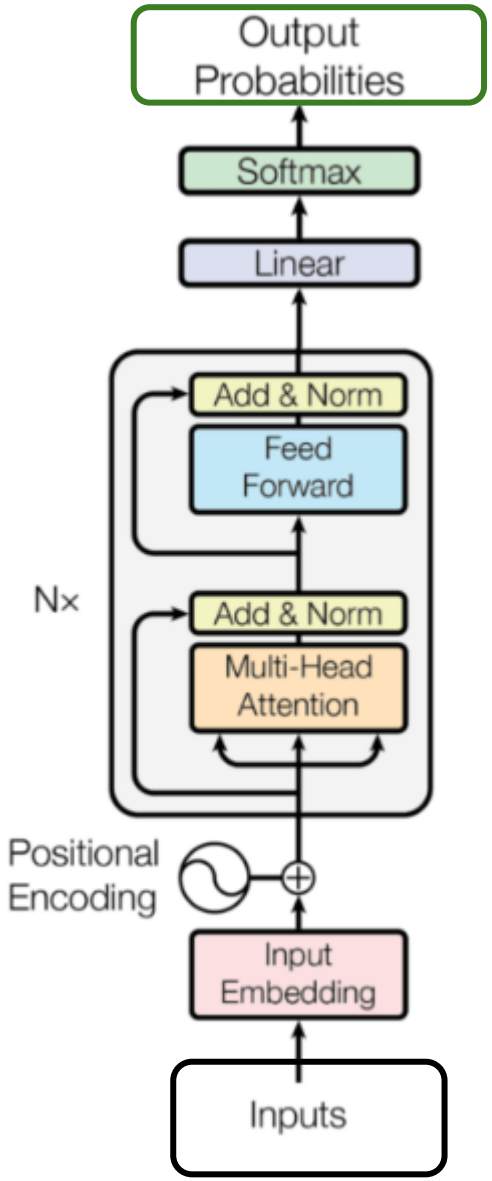
I | ordered | a | pizza | with | peperoni | and | cheese | .

What information did you use to make that prediction?

Language Model

I | ordered | a | [MASK] | with | peperoni | and | cheese | .

Input:
(sequence of) tokens



Tokenization

corpus: frequency counts

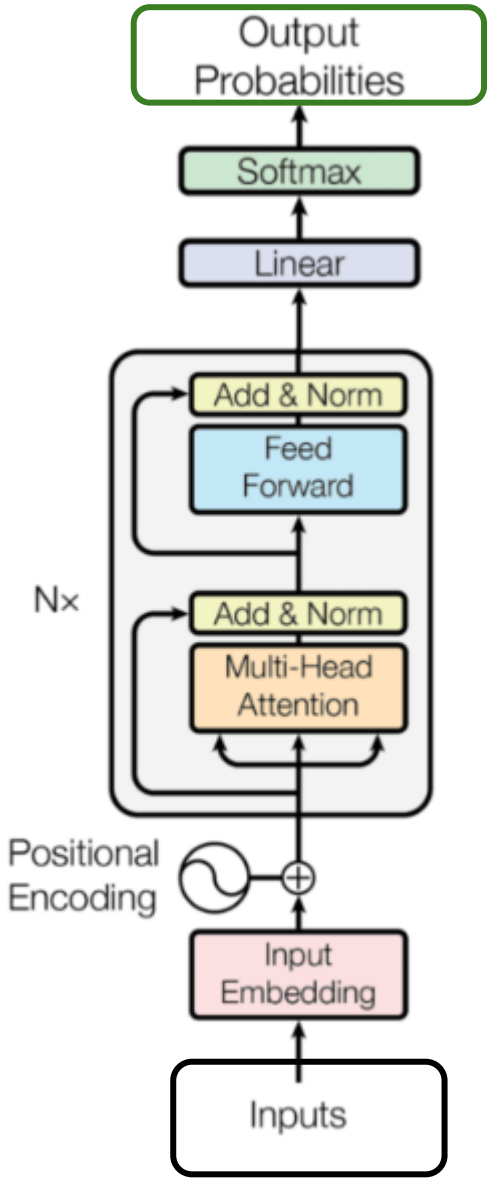
```
("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)
```

vocabulary:

- merge most frequent pair of smaller units
- add to vocabulary
- repeat

... after 3 steps

```
Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug", "un", "hug"]
Corpus: ("hug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("hug" "s", 5)
```



https://huggingface.co/learn/llm-course/en/chapter6/5

Positional encoding

To each input embedding, add a position-specific embedding (position_1, position_2, …., position_n)

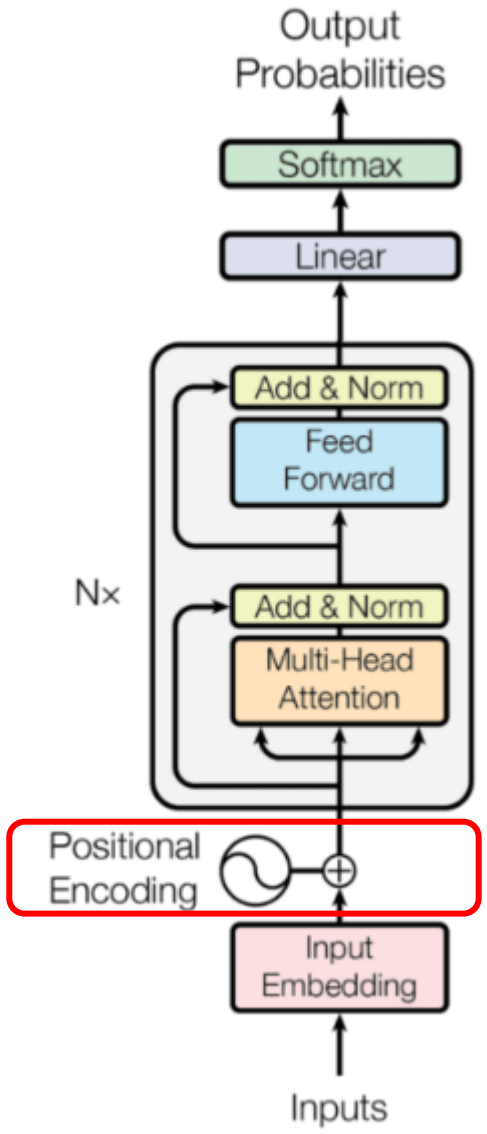
[the] [student] [aced] [the] [ ]

$$\overrightarrow{[\text{the}]} + \overrightarrow{\text{position 1}}$$

$$\overrightarrow{[\text{student}]} + \overrightarrow{\text{position 2}}$$

$$\overrightarrow{[\text{aced}]} + \overrightarrow{\text{position 3}}$$

$$\overrightarrow{[\text{the}]} + \overrightarrow{\text{position 4}}$$



https://huggingface.co/learn/llm-course/en/chapter6/5

Self-attention

Attention

Output:
Masked token

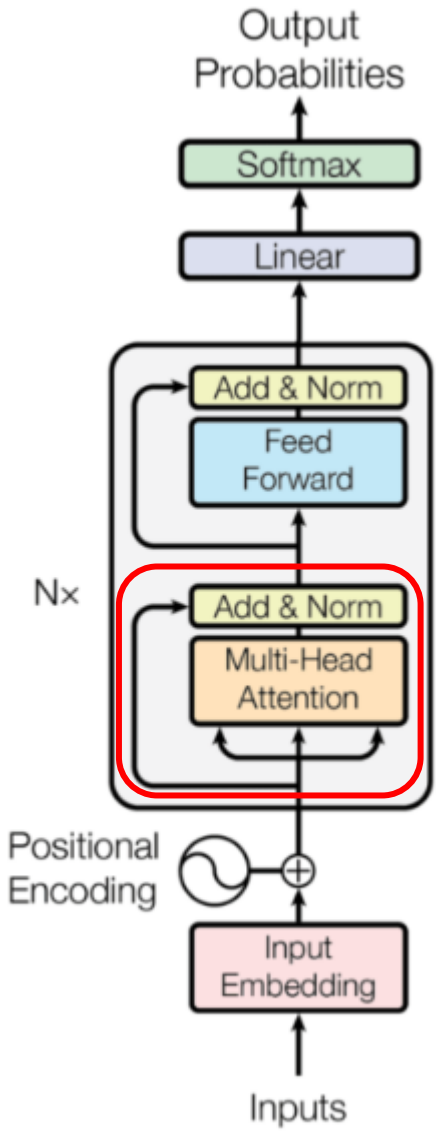
I | ordered | a | pizza | with | peperoni | and | cheese | .

What information did you use to make that prediction?

Language Model

I | ordered | a | [MASK] | with | peperoni | and | cheese | .

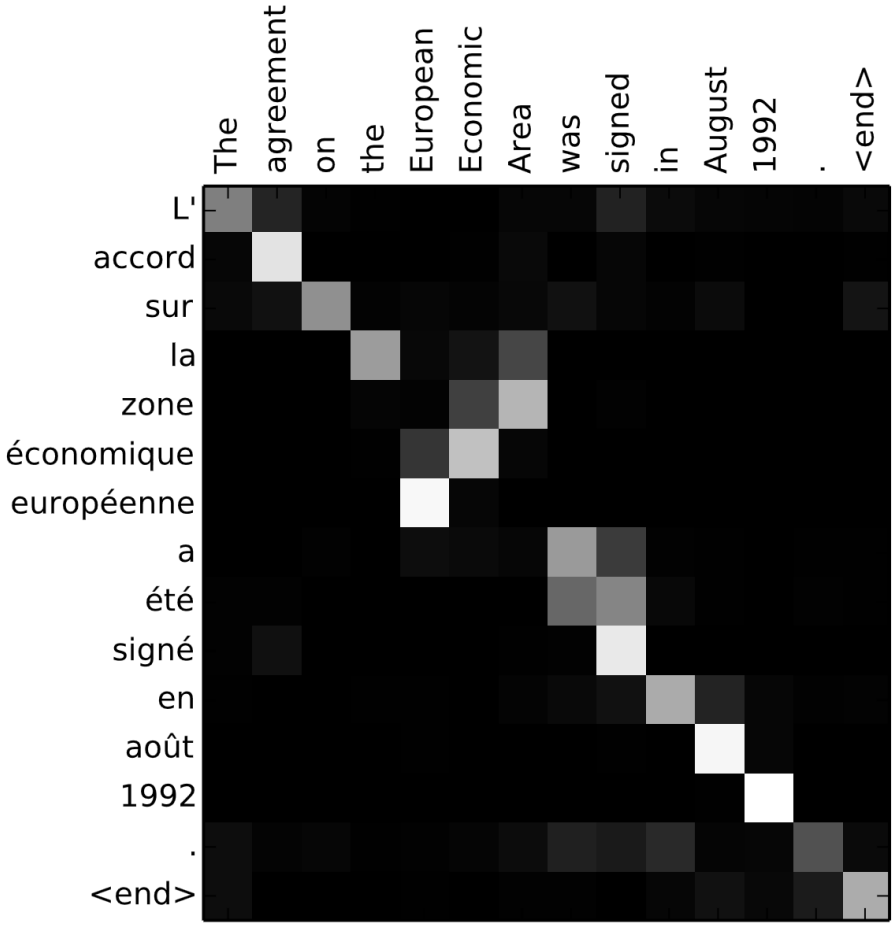
Input:
(sequence of) tokens



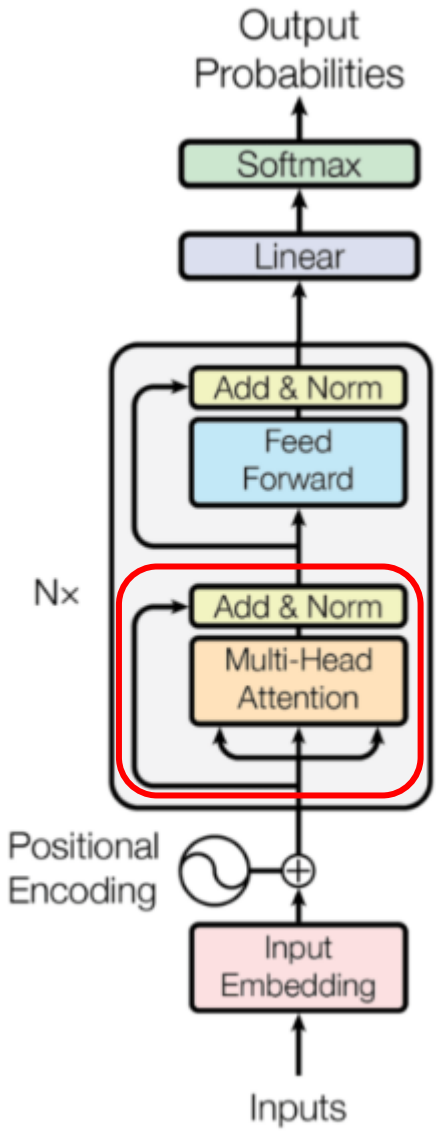
Self-attention

Attention

Originally developed for machine translation



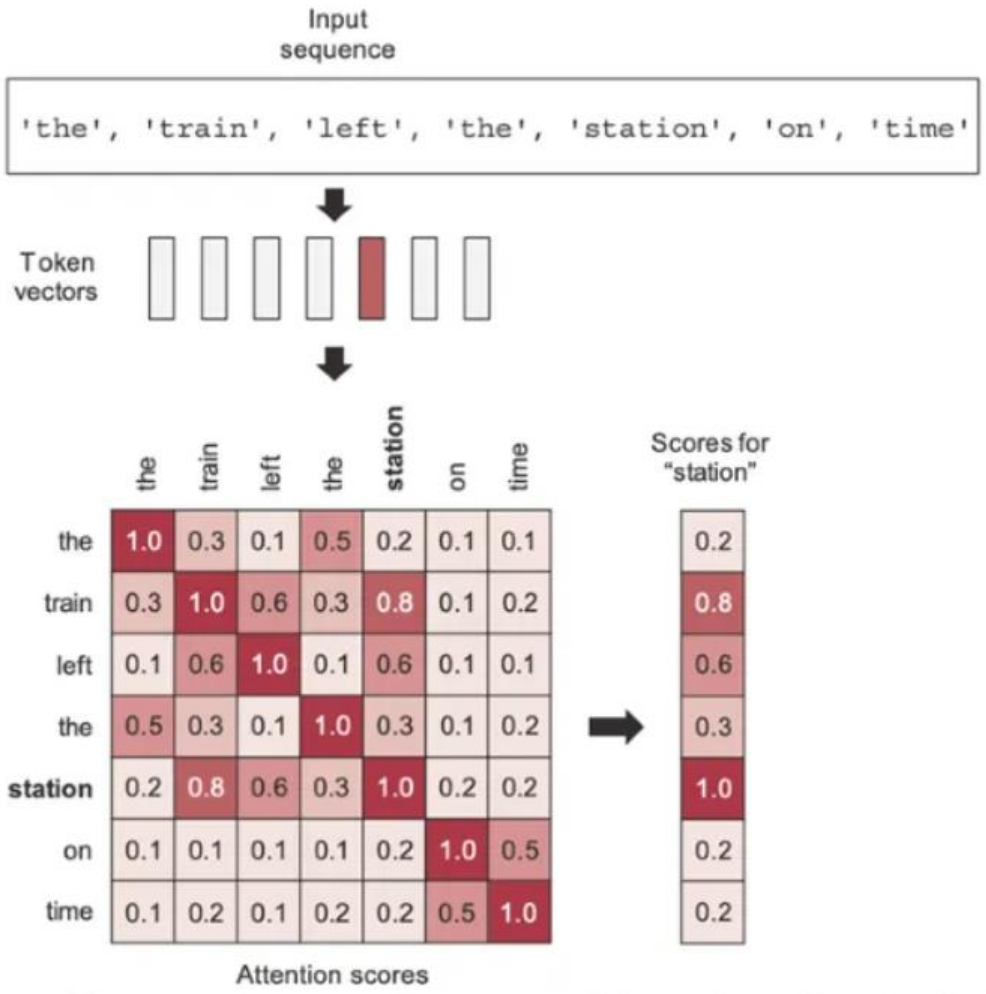
Bahdanau et al., 2015

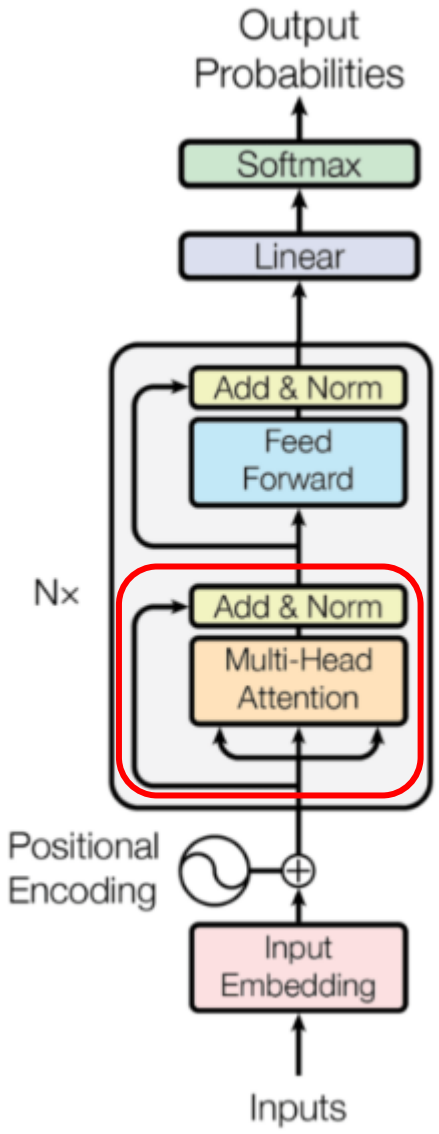


Self-attention

Self-Attention

Use attention just within the input to predict tokens

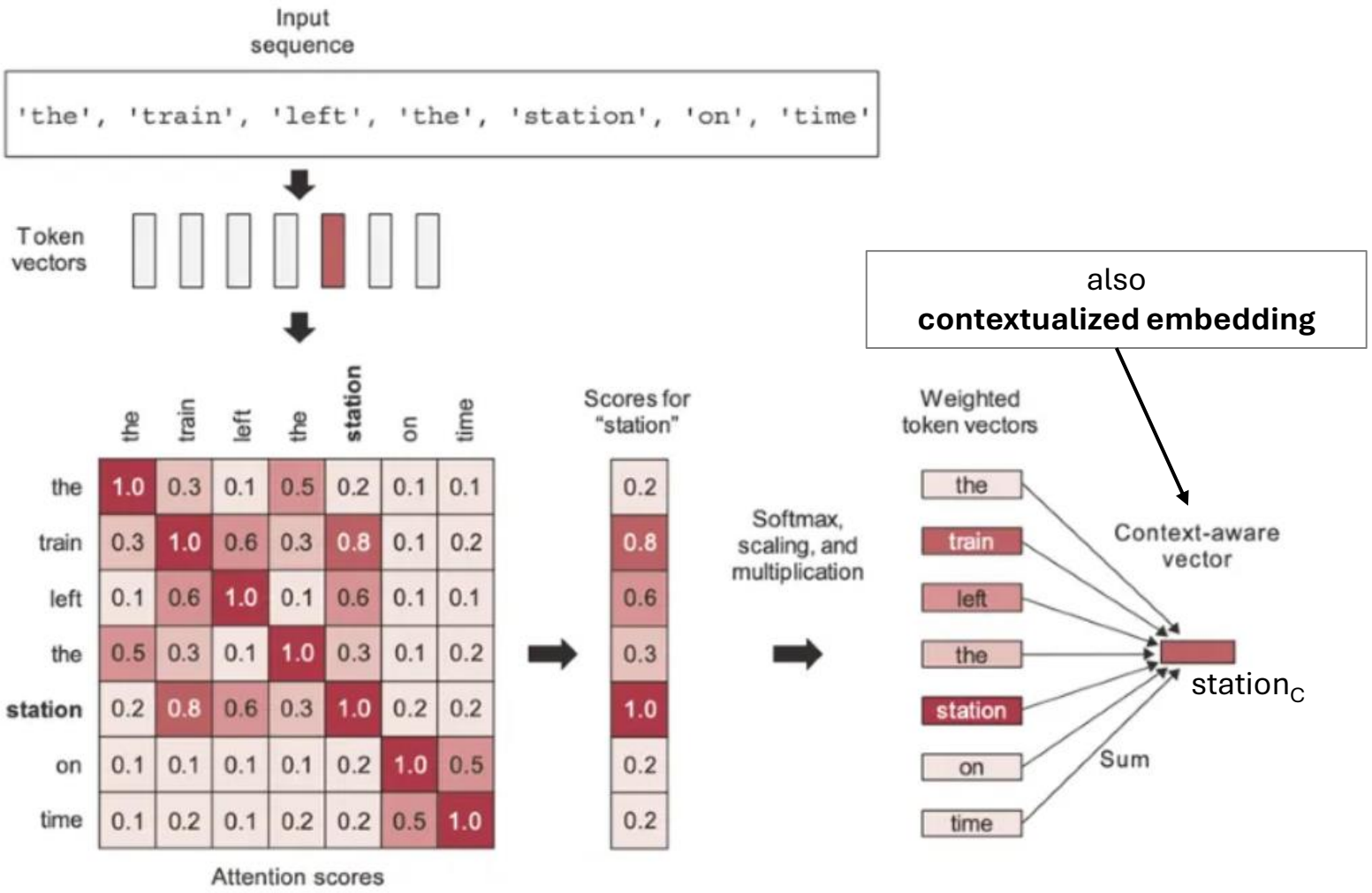


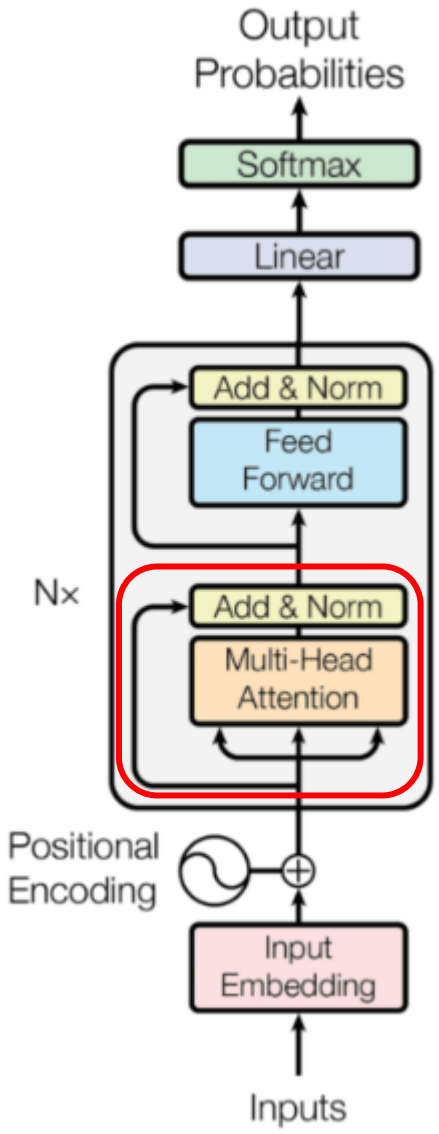


https://medium.com/@ramendrakumar/self-attention-d8196b9e9143

Self-Attention

Use attention just within the input to predict tokens

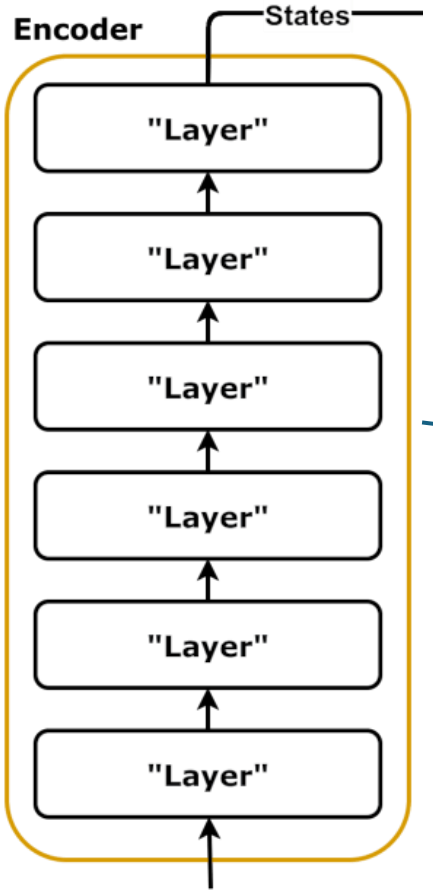


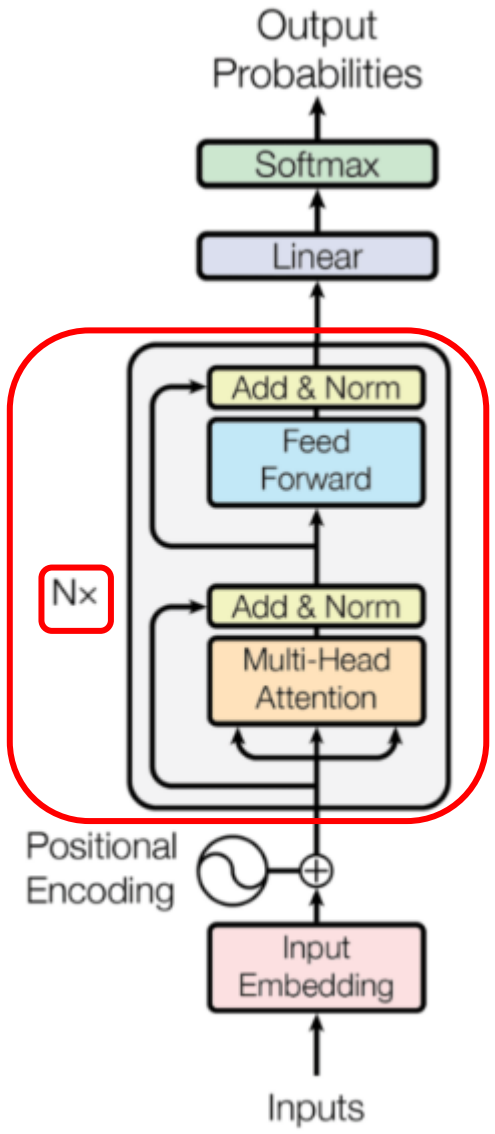


https://medium.com/@ramendrakumar/self-attention-d8196b9e9143

Self-attention

Multiple attention layers





https://medium.com/@ramendrakumar/self-attention-d8196b9e9143

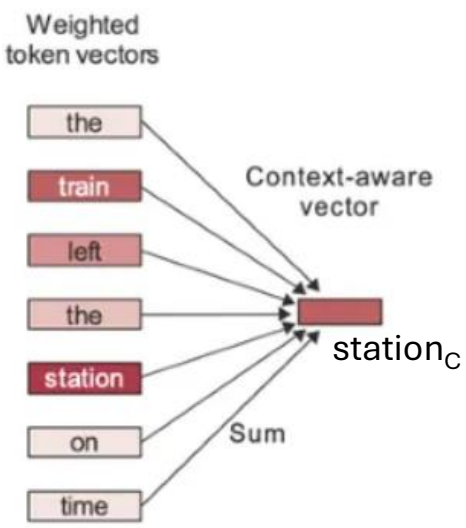
Self-attention

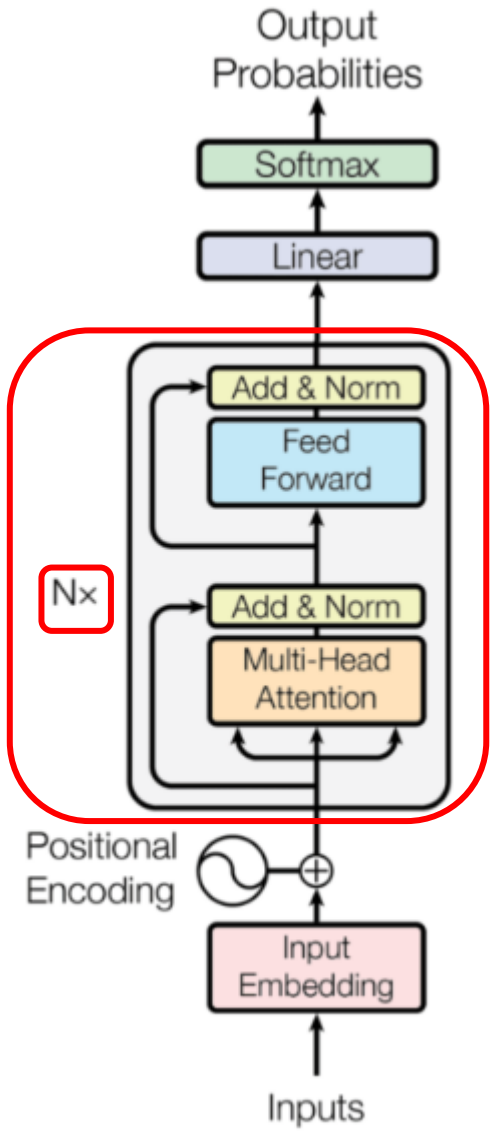
Multiple attention layers

Repeat this N times:

- Layer 1: Input embeddings as input
- Layer 2: Contextualized embeddings layer 1 as input
- Layer 3: Contextualized embeddings layer 2 as input
- ...

Model Name	n_{params}	n_{layers}
GPT-3 Small	125M	12
GPT-3 Medium	350M	24
GPT-3 Large	760M	24
GPT-3 XL	1.3B	24
GPT-3 2.7B	2.7B	32
GPT-3 6.7B	6.7B	32
GPT-3 13B	13.0B	40
GPT-3 175B	175.0B	96





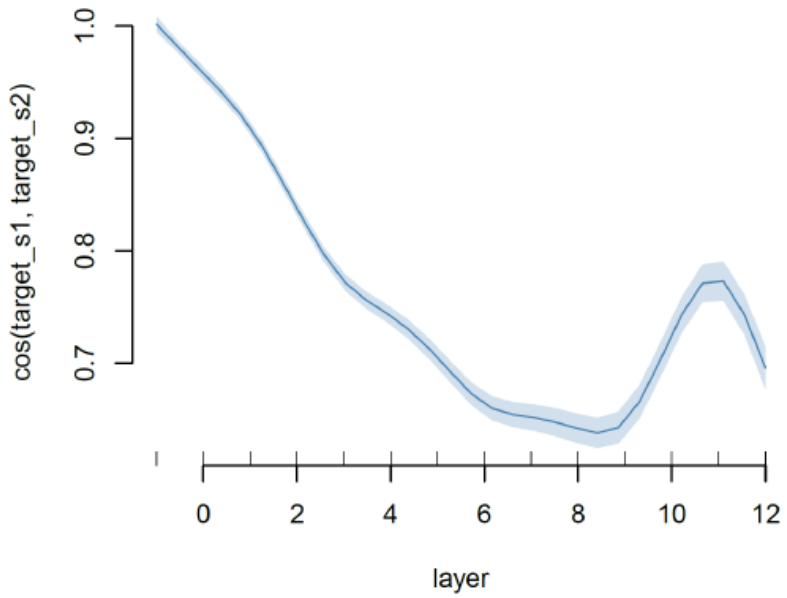
https://medium.com/@ramendrakumar/self-attention-d8196b9e9143

Self-attention

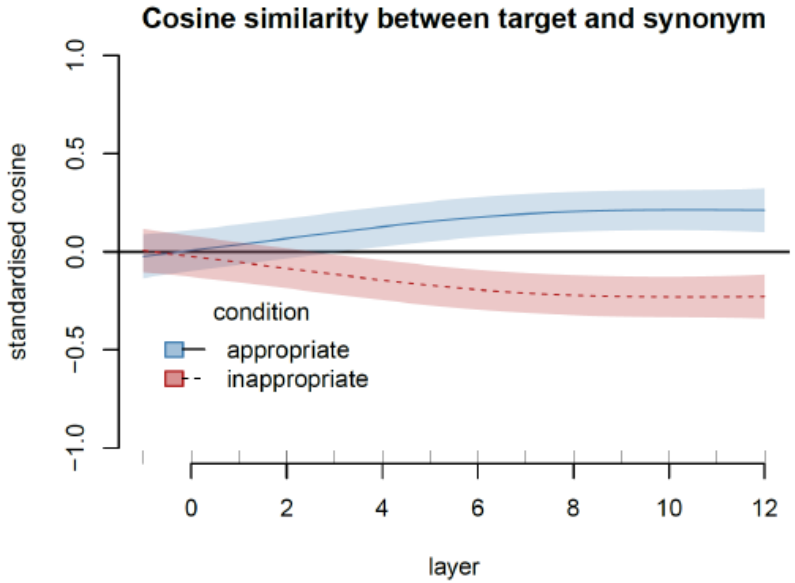
Multiple attention layers

sentence	appropriate	inappropriate
The player swung but missed the ball.	sphere	party
The girl dressed up to attend the ball.	party	sphere

(a) *same target, different sense*



(b) *similarity by condition*



Contextualized embeddings drift apart over layers

Cassani, Bianchi, Attanasio, Marelli, & Günther (2023)

Large Language Models: Transformer overview

BERT*

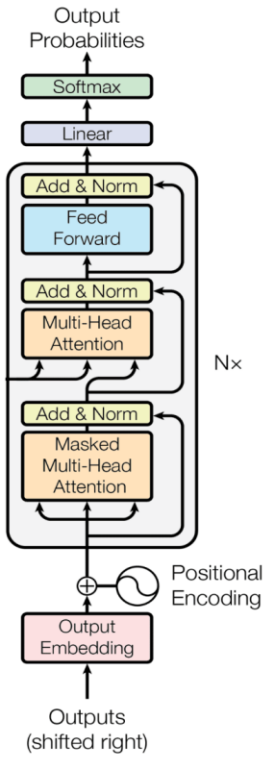
Encoder-only

Get accurate representation of input

Transformer

Encoder **Decoder**

GPT*



Decoder-only

Generate text

*actual model architectures differ in details

Decoder: Training objective

Output:
Next token t

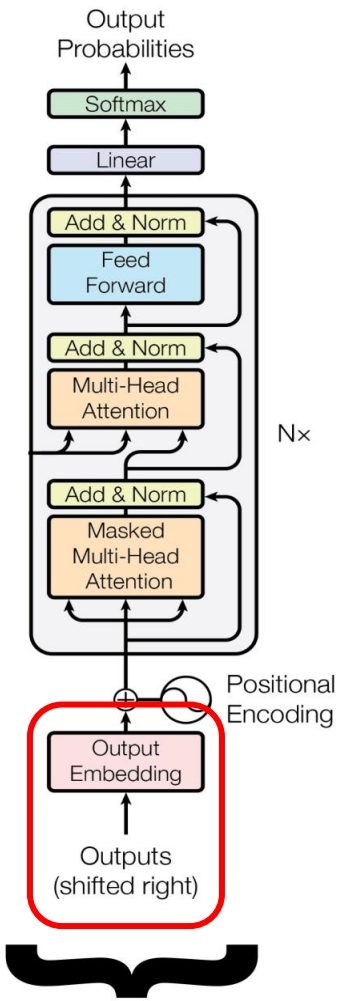
I | ordered | a | pizza | with | peperoni | and | cheese | .

Language Model

I | ordered | a | pizza | with | peperoni | and | cheese | .

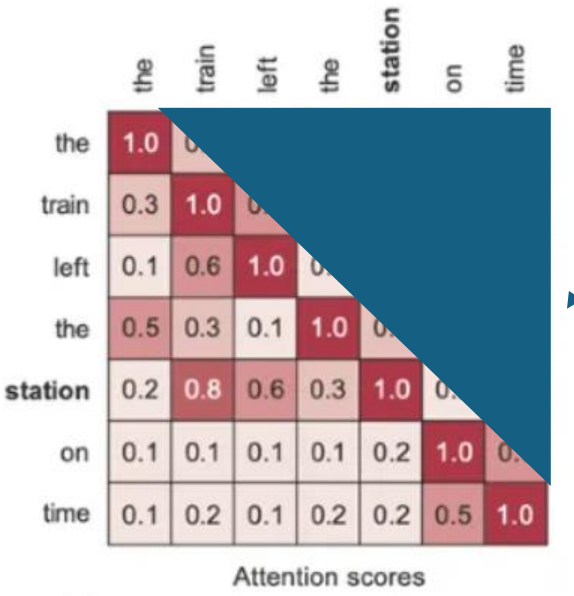
Input: previous tokens 1, ..., t-1

GPT*



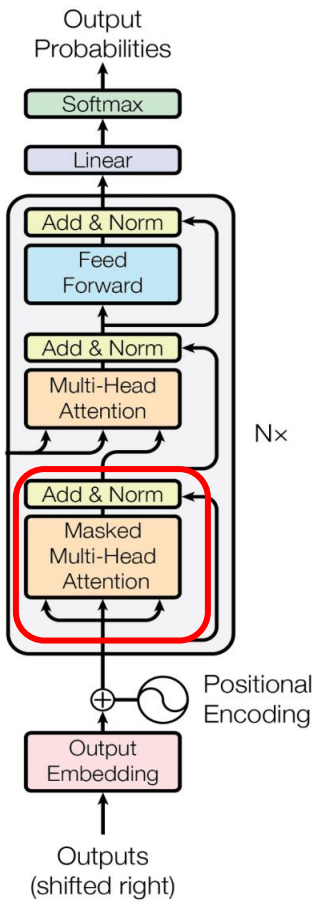
Decoder-only

Masked attention



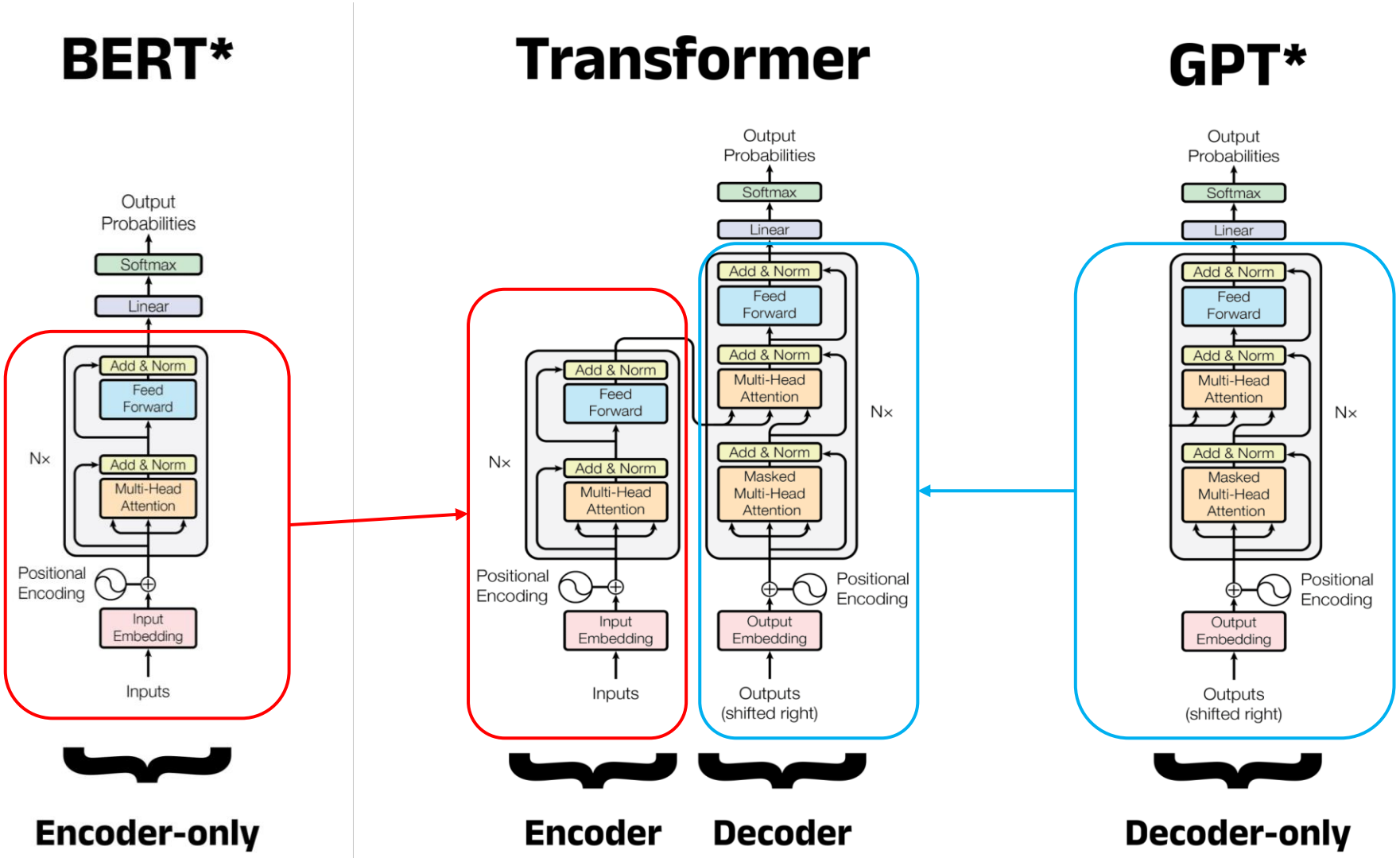
No access to the future when predicting the next word

GPT*



Decoder-only

Large Language Models: Transformer overview



Predict the **correct** word

Predict the **next** word

*actual model architectures differ in details

Large Language Models: Extensions

- The chatbots we use today (ChatGPT,…) are no longer pure language models (their output can be *wild*)

- Some additional components:
  - **Fine-tuning** (pre-train large model, then adjust weights on specific corpus)
  - **RLHF** (reinforcement learning by human feedback)
  - **System prompts** (implicitly, start each session with a standardized prompt that ensures certain „behavior“/“personality“)
  - **Chain-of-thought** (model articulates intermediate reasoning steps, „Thinking…“)
  - **Multilingual** (trained on corpora from different languages)
  - **Multimodal** (trained on text + images)

- Here, we won‘t get into those
- Also, many proprietary models are not open (i.e., model internals like weights, embeddings, probabilities etc. are not freely available)

Summary

Summary

- Language Models learn to predict tokens from context (what „token“ and „context“ mean exactly can differ), and thus model $P(\text{token}|\ \text{context})$

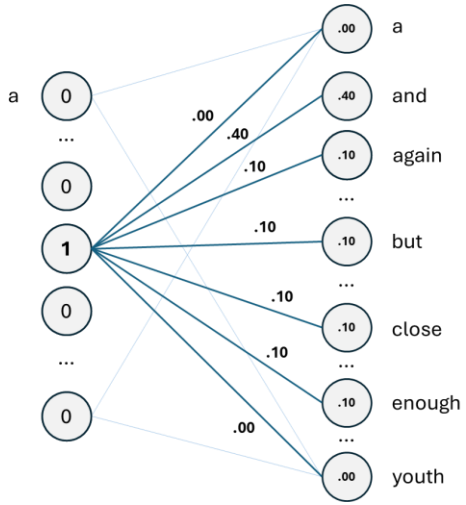
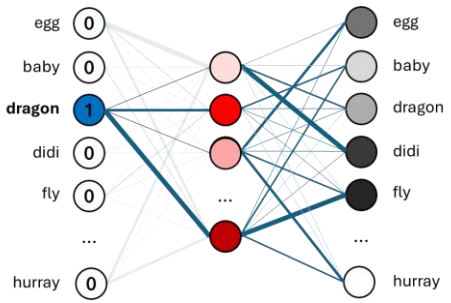
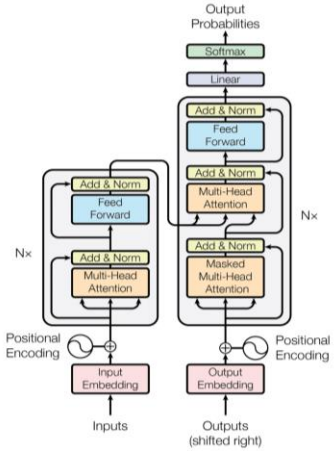
- **Any** language model produces
  1) **Output** by **prompting** (= providing start tokens)
  2) **Probabilities/surprisal** for tokens and token sequences
     - $P(\text{token}_1, \ldots, \text{token}_n|\ \text{context})$
     - $P(\text{token}_1, \ldots, \text{token}_n)$

- Neural langauge models also produce
  3) **Embeddings (= vectors)** for [sequences of] tokens
     - Single-layer model: One **static embedding**
     - LLMs (transformer): Increasingly **contextualized embeddings**

1

the other birds complained that ever so suddenly spread out of sight but why ? <END>

2



3